



MASTER THESIS

In Order to Obtain the

RESEARCH MASTER

in

Data Science

Presented and defended by:

Ali Alzein

On,, 2026

Title

LLM-Enhanced Graph-Based Intrusion Detection

Supervisors

Dr. Hussein Hazimeh

Dr. Fouad Al Tfaily

Reviewers

Dr. Hassan Harb

Dr. Ali Jaber

Abstract

Internet of Things (IoT) devices generate network traffic whose interpretation depends on both individual connections and the communication patterns around them. Conventional machine learning can classify connections from numerical measurements, while deep learning can learn richer representations of those measurements. Graph neural networks (GNNs) add communication context by representing devices as nodes and connections as edges. Pretrained language models provide another view by encoding textual descriptions of network events. These views motivate combining structural and semantic evidence, although combining models does not by itself establish that each interaction improves detection. This report presents GLASS (Graph-Language Adaptive Semantic-Structural fusion), an edge-level intrusion-detection pipeline combining a graph attention network with a frozen cybersecurity language encoder. GLASS describes each aggregated flow using a fixed text template, combines graph and language embeddings through a gate that varies across edges and feature dimensions, and evaluates an iterative feedback stage that consults a semantic classifier on uncertain edges. The evaluation compares a prototype semantic scorer and a trained classification head, including the head alone, using pooled out-of-fold predictions across three training seeds. Experimental results on the NF-UNSW-NB15 and NF-ToN-IoT NetFlow datasets show that the trained-head feedback configuration improves over the GNN, prototype semantic and fusion stages by statistically separated margins on both datasets. However, the trained head alone has slightly higher mean macro-F1, and its difference from feedback is not separated. Fusion improves over the prototype semantic model on both datasets, but does not establish an improvement over the GNN. With output fusion disabled at the earlier consultation settings tested, neither realistic consultant provides a separated injection gain, and trained-head injection produces a separated loss on NF-ToN-IoT; an oracle supplied with true labels improves through the same channel. GLASS therefore provides evidence for the value of a trained semantic classifier and a controlled way to examine graph–language interaction, while an additional benefit from feedback remains unestablished. The findings concern label-conditioned aggregated graphs and do not estimate deployment performance.

Keywords: GLASS, intrusion detection, graph neural networks, edge classification, structural–semantic fusion, uncertainty-guided feedback, negative results.

Contents

List of Figures	4
List of Tables	5
1 Introduction and Context	6
1.1 Why combine graph and language representations?	8
1.2 Research problem and questions	9
1.3 Objectives and contributions	10
1.4 Report structure	11
2 Related Work	12
2.1 Graph-based network intrusion detection	12
2.2 Language models over security text and network traffic	13
2.3 Fusing structural and semantic representations	13
2.4 GNN and LLM combinations in intrusion detection	15
2.5 Iterative GNN–LLM feedback	16
2.6 Positioning	17
3 Method	19
3.1 Datasets and Common Processing Pipeline	19
3.2 Directed Multigraph Construction	19
3.3 Edge Feature Encoding	22
3.4 Structural Branch	22
3.5 Semantic Branch	23
3.6 Adaptive Gated Attention Fusion	25
3.7 Uncertainty-Guided Semantic Feedback	26
3.8 Baseline Protocol	29
3.9 Evaluation and Training Protocol	31
4 Results	33
4.1 The protocol as it was run	33
4.2 The model ladder	33
4.3 Feedback ablations and the mechanism in isolation	37
4.4 Five attempts on the embedding-only consultant	39
4.5 Attention injection	40
4.6 Re-trained published baselines	40
4.7 Per-class behaviour	44
4.8 Knob selection	47
5 Discussion	48

5.1	Answers to the research questions	48
5.2	Patterns across the two imbalance regimes	49
5.3	Where the feedback model’s gain comes from	50
5.4	What the configuration buys, and what it does not	50
5.5	Limitations	51
5.6	What follows from this	52
5.7	Scope, and the experiments this work did not reach	53
6	Conclusion	54
	Acknowledgements	56
	Statement on the use of AI assistance	56
	Data and code availability	56
	References	57
A	Negative mechanism results	61
A.1	Conditions A, B and C	61
A.2	What the canonical feedback model was actually injecting	62
A.3	Per-class reliability of the embedding-only consultant	62
A.4	The five treatments	64
A.5	Isolation diagnostics and prediction changes	64
B	Supporting tables	66
B.1	Per-class F1	66
B.2	Knob selection curves	66
B.3	Consultant complementarity on the flagged set	67
B.4	Model comparisons and metric interpretation	68
B.5	Baseline scoring asymmetry and comparison details	69
C	Reproducibility	71

List of Figures

1	Pipeline architecture and evaluated stages	20
2	Class sizes on the two aggregated graphs	21
3	Selection, injection and output fusion in the feedback model	27
4	Four-stage model comparison under both consultants	35
5	Macro-F1 and accuracy by model stage	36
6	Bootstrap intervals for the stage comparisons	37
7	Injection diagnostic with output fusion disabled	39
8	Endpoint ambiguity and E-GraphSAGE performance	41
9	Point-estimate decomposition of the NF-UNSW-NB15 gap	43
10	Per-class F1 by model stage	45
11	Per-class feedback differences	46

List of Tables

1	Per-variant edge focus weights	23
2	Baseline hyperparameters used in the reported runs	30
3	Pooled out-of-fold macro-F1 by model stage	34
4	Model and component comparison on three metrics	35
5	Bootstrap comparisons between model stages	36
6	Complete-system feedback ablations	38
7	Injection mechanism measured in isolation	38
8	Re-trained baseline macro-F1	41
9	Feedback comparisons against re-trained baselines	42
10	Decomposition of the NF-UNSW-NB15 baseline gap	42
11	Pooled out-of-fold macro-F1, mean over three seeds, by condition . . .	61
12	Injected bias magnitude on flagged edges, per seed, by condition	62
13	Prototype precision on training edges	63
14	Five prototype-consultant treatments	64
15	Per-class F1 and edge counts	66
16	Feedback selection curves	67
17	Consultant accuracy on uncertain edges	67
18	Consultant correctness on disagreements	67
19	Committed artifacts behind the report’s numbers	72

1 Introduction and Context

This section first introduces the intrusion-detection problem and the scope of the study. It then explains why graph and language representations are combined, states the research questions and contributions, and finally outlines the report.

Internet of Things (IoT) networks connect sensors, cameras, appliances and industrial controllers. Their traffic must be monitored for attacks while legitimate services continue to operate. Missed attacks can leave harmful activity unchallenged, while false alarms consume analyst attention. Alert validation and threat prioritisation are therefore practical requirements of intrusion monitoring [29].

Conventional machine learning provides a way to classify traffic from measurements such as duration, byte count, protocol and destination port. When each connection is represented as an independent row, its surrounding communication pattern is not represented directly. Deep learning can learn richer feature representations, but a model operating on individual rows still needs an explicit way to use relationships between connections. This motivates moving from isolated traffic records to a graph representation.

Graph neural networks (GNNs) represent communicating devices as nodes and their connections as edges. They can combine a connection’s attributes with information from neighbouring devices and links, as in graph-based intrusion detectors such as E-GraphSAGE [17]. This context is relevant when an attack is expressed across many connections: a scan may contact many hosts, while a distributed attack may converge on one destination. Graph structure supplies a view that an isolated flow record does not directly contain.

Pretrained language models offer a complementary representation of an event described in text. Large language models (LLMs) can generate explanations, while encoder models such as BERT produce representations that can support classification [6]. This study uses CySecBERT, a cybersecurity-adapted BERT encoder, to represent short templated descriptions of traffic [3]. It remains frozen and does not generate text. Its embeddings are another representation of the recorded flow attributes, not an independent source of observations.

Previous work has combined graph and language models in several ways. Parallel encoders with attention-based fusion have been studied in misinformation detection [21]. In intrusion detection, XG-NID uses a GNN for classification and a generative LLM for explanations and remedial guidance [8]. DAS already uses iterative graph-language feedback for node-level learning [32]. Thus, neither fusion nor iterative feedback is new by itself; their value in the present edge-level detection task must be established through comparison and diagnostic tests.

This report develops GLASS (Graph-Language Adaptive Semantic-Structural fusion). GLASS represents aggregated flows as directed edges, learns their structural context through a graph attention network, encodes their text descriptions with frozen CySecBERT, and combines the embeddings using a gate that varies by edge and feature dimension. Its feedback stage consults a semantic classifier on uncertain edges and injects the advice before another graph update. We evaluate the structural, prototype semantic, fusion and feedback stages, compare prototype and trained-head consultants, and measure the trained head alone. Tests that disable output fusion examine whether injection itself helps. The same pipeline is evaluated on NF-UNSW-NB15 and NF-ToN-IoT under the graph-construction assumptions in Section 3.2; the scores are controlled comparisons, not deployment estimates.

NF-UNSW-NB15 derives from UNSW-NB15, whose original benchmark was created from a mixture of normal activity and contemporary attacks in the UNSW Canberra Cyber Range Laboratory [23]. NF-ToN-IoT derives from the TON_IoT datasets and provides an IoT-focused telemetry setting [22]. This study uses both in their NetFlow-standardised forms [28]. One identical processing and modelling pipeline is applied to the two datasets. Every architectural choice is shared between them. Only two quantities selected on validation data differ.

The two datasets were chosen because they occupy very different points on the class-imbalance axis. After aggregation, NF-UNSW-NB15 contains 656 edges, is 47.4% benign, and has a most-to-least-frequent class ratio of 12.4:1. Eight of its nine attack classes contain 40 edges and the ninth contains 25, so only one falls at or below 35 edges. NF-ToN-IoT contains 2,127 edges, is 82.1% benign, and has a most-to-least-frequent class ratio of 582:1. Seven of its nine attack classes contain 35 edges or fewer. Its distribution is far more skewed. By the most-to-least class ratio, the two graphs differ by roughly a factor of 47 in imbalance severity. That ratio counts the two NF-ToN-IoT classes the metric excludes, `dos` and `ransomware`; over the classes the metric scores it is 145.5:1, still roughly twelve times NF-UNSW-NB15’s. The model faces the whole graph, so this report quotes the graph-level ratio, and the two descriptors that do not depend on that choice point the same way: 82.1% of NF-ToN-IoT’s edges are benign against 47.4%, and seven of its nine attack classes hold 35 edges or fewer against one.

That difference is the methodological reason for using two datasets. A single dataset cannot separate a property of the architecture from a property of the class distribution that architecture happened to face. Holding the architecture fixed while changing the imbalance regime by roughly $47\times$ makes that distinction experimentally addressable. The two graphs are not matched on everything else, and we do not claim they are. They also differ in the share of edges that repeat an ordered address pair, 58.7% against 7.9%, and in size and density, at 49 nodes and 656 edges against 1,501 and 2,127. Two datasets

cannot separate those axes from imbalance, and Section 4.5 uses them in their own right. RQ4 already asks how class imbalance affects fusion and feedback, and the two-dataset design gives that question two substantially different regimes to compare. No stage is assumed in advance to be sensitive or insensitive to that change.

The scoring protocol also differs in one necessary respect. Macro-F1 covers ten classes on NF-UNSW-NB15 and eight classes on NF-ToN-IoT, because two NF-ToN-IoT classes are too small to survive five-fold splitting. Their edges remain in the graph for message passing but are excluded from the metric. Because the class counts differ, absolute scores are not comparable across the two datasets. Only the shape of the model ladder is comparable, that ladder being the four stages of this study placed in order, each compared against the one below it. This constraint applies throughout the report.

1.1 Why combine graph and language representations?

Graph neural networks (GNNs) provide a direct way to represent communication structure. In this work, IP addresses form the nodes of a directed graph and aggregated flows form labelled edges. A graph-attention encoder propagates information between communicating hosts and produces a structural representation for each edge. This branch can learn patterns that are difficult to express as independent rows in a conventional feature table. It does not, however, possess an explicit vocabulary for interpreting the traffic. The graph tells the model *who communicated with whom* and how that interaction is positioned in the network. It does not explain what a high-frequency TCP connection to a particular service may signify.

A pretrained cybersecurity language encoder offers a complementary view. The same aggregated flow can be written as a short, controlled sentence describing its endpoints, protocol, port, frequency, volume, and duration. The implementation uses CySecBERT, a BERT-class model adapted to cybersecurity text and evaluated by its authors on domain-specific representation and classification tasks [3]. It is frozen and used purely as a sentence encoder. Throughout this report, this component is called the *semantic branch*. That terminology distinguishes its representational role from that of a generative large language model. Where the same component is evaluated on its own as one stage of the comparison, it is called the semantic model.

Neither representation is assumed to win in every case; the design instead lets them challenge and complement one another. The parallel fusion stage runs the graph and semantic encoders side by side and combines their edge embeddings through Adaptive Gated Attention Fusion, referred to throughout as the fusion model. The feedback stage adds uncertainty-guided semantic feedback: edges with high-entropy GNN pre-

dictions are selected for semantic consultation, and the resulting signal is reintroduced by modifying the edge representations that the next graph update consumes. A second design, which biases graph attention directly, was also implemented and measured. It produced no change in prediction at all, and is reported as a verified negative result.

The coupling is bidirectional in routing. The graph model’s per-edge uncertainty decides which edges are consulted, while the semantic branch returns a per-edge judgement that re-enters the edge representations consumed by the next graph update. It is not bidirectional in content: the consultant’s judgement depends on that edge’s text alone, not on the graph model’s current belief about the edge.

The distinction from the prior work introduced above is the edge-level intrusion-detection setting and its controlled evaluation. GLASS tests sample-adaptive fusion and uncertainty-guided consultation, then separates the performance of the complete configuration from that of its semantic consultant and injection path. Section 2 develops the comparison with earlier systems.

1.2 Research problem and questions

The research problem is not simply whether adding a language embedding raises a score. If one branch is already a strong classifier, a fusion module can appear successful while doing little more than copying that branch. Likewise, a feedback loop can improve the final output because semantic features are fused at the classifier, even when the claimed attention re-entry mechanism contributes nothing. The evaluation must separate the value of the two modalities, the fusion mechanism, the uncertainty-selection policy, and the iterative update itself.

Five research questions frame the study:

- RQ1.** How do the structural GNN branch and the semantic branch perform individually on NF-UNSW-NB15 and NF-ToN-IoT under the same five-fold out-of-fold evaluation protocol?
- RQ2.** Does the fusion model score above both single-branch rungs on NF-UNSW-NB15 and NF-ToN-IoT, and are those differences separated under the bootstrap protocol, meaning that their intervals exclude zero?
- RQ3.** Does uncertainty-guided semantic feedback add value beyond static fusion on NF-UNSW-NB15 and NF-ToN-IoT? If so, does the gain come from semantic re-entry into the graph representation or from final-output fusion?

RQ4. How do graph topology, repeated endpoint pairs, class imbalance, and the strength of the semantic consultant affect fusion and feedback on NF-UNSW-NB15 and NF-ToN-IoT?

RQ5. How does the feedback model compare against published graph-based intrusion detection architectures re-trained on the same aggregated representation under the same protocol on NF-UNSW-NB15 and NF-ToN-IoT, and how much of any observed difference is attributable to representation and tuning rather than to architecture?

These questions deliberately allow a negative answer. The experiments are meant to determine when the proposed interaction works, when it does not, and why. A model ladder of GNN, semantic encoder, fusion and then feedback is only meaningful if each higher rung is compared fairly with the strongest relevant lower rung.

1.3 Objectives and contributions

The practical objective of GLASS is to build one reproducible pipeline that transforms NF-UNSW-NB15 and NF-ToN-IoT NetFlow records into two aligned views and evaluates every stage under one consistent protocol. The work makes five contributions.

1. It implements a directed, edge-classification graph pipeline in which flows are aggregated by source IP, destination IP, and attack class, with centrality-based node features and statistical edge attributes.
2. It serialises each edge as a controlled-language description using a fixed sentence template. The text supplied to the encoder is kept label-free by an explicit validation guard, preventing the semantic branch from seeing the target attack name or a synonymous relation label.
3. It implements and compares two multimodal stages: the fusion model for parallel structural– semantic fusion, and uncertainty-guided semantic feedback that reintroduces semantic evidence into the edge representations of uncertain edges. The feedback model consults a classification head trained on the semantic embeddings. The feedback model is reported under that consultant and under the weaker embedding-only scorer it was first built with, so the reader can see how much of the result is the mechanism and how much is the consultant.
4. It provides a controlled evaluation on NF-UNSW-NB15 and NF-ToN-IoT, including unimodal baselines, fusion comparisons, uncertainty-selection ablations, a comparison between the two consultants, mechanism-isolation tests, and two

published graph-based intrusion detection architectures re-trained on the same representation. It reports both positive and negative results to show which components carry the measured gain, including five measured attempts to make an embedding-only consultant’s advice usable by the feedback channel, none of which succeeded.

5. It provides a cross-dataset comparison under one shared architecture across two substantially different class-imbalance regimes, testing whether the behaviour of each stage in the model ladder is a property of the method or of the class distribution it faces.

The central result is that a strong semantic classifier explains much of the complete system’s performance, while an added benefit from feedback injection remains unestablished. The trained-head feedback configuration is separated above the GNN, semantic and fusion stages and the faithful re-trained baselines on both datasets. However, the trained head alone has a slightly higher mean macro-F1 on both, with no separated difference from feedback. The prototype-feedback configuration does not establish a gain over either structural stage. Changing the consultant also changed the consultation percentile, so the increase between configurations cannot be assigned to the consultant alone.

The mechanism tests make this distinction explicit. With output fusion disabled, neither realistic consultant gives a separated improvement through injection at the earlier consultation settings tested; the trained head produces a separated loss on NF-ToN-IoT. An oracle with access to the true label does improve through the same path. Thus the channel can carry useful information, but the tested consultants do not establish an injection benefit. The contribution is the implementation and controlled evaluation of this design, including these negative results and the limits of their interpretation.

1.4 Report structure

The following sections review related detection and fusion methods (Section 2), describe GLASS and its evaluation protocol (Section 3), present model comparisons and diagnostic results (Section 4), interpret findings and discuss their limitations (Section 5), and conclude with findings and future directions (Section 6); the appendices provide supporting measurements and reproducibility details.

2 Related Work

This section positions GLASS within earlier work. It first reviews graph-based intrusion detection and language models for security data. It then examines how structural and semantic representations are fused, reviews graph–language feedback mechanisms, and finally explains how this study relates to that work.

2.1 Graph-based network intrusion detection

Intrusion detection is conventionally divided by detection methodology. The guidance this report follows names three primary classes, signature-based, anomaly-based and stateful protocol analysis [29], and the first two can both be posed over a flow graph. A flow graph makes the edge the natural prediction unit when the object to be classified is a communication event between two endpoints. Nodes represent communicating entities such as IP addresses, and edges represent flows or aggregated flows. This formulation preserves the relational context around a connection without changing the classification target: the decision remains edge-level. It also separates information attached to the communicating endpoints from information carried by the flow itself.

E-GraphSAGE is an important reference point for this formulation. It applies a GNN to intrusion detection and classifies an edge from the concatenation of the embeddings of its two endpoints [17]. The edge’s own features do not reach the classifier. That choice gives the classification head a purely endpoint-derived view of the connection. A direct consequence is that two parallel edges with the same ordered endpoints present the same pair of endpoint embeddings to the edge classifier, even if the edges differ in attributes that are not otherwise represented at the head. The architecture can express structural context around an endpoint pair, but its final edge representation does not preserve an explicit channel for the attributes of the particular edge being classified.

TE-G-SAGE moves to an edge-aware GraphSAGE formulation: its supervised edge head consumes the edge’s own features, and the model is paired with SHAP-based explanation, an attribution method that divides a prediction into per-feature contributions [20]. This changes the representational question. Once the edge itself is allowed to carry predictive information, the model can distinguish endpoint context from flow-specific attributes rather than requiring the classifier to infer the latter indirectly from node embeddings.

For network-flow intrusion detection that distinction matters, because the graph and the individual connection encode different kinds of evidence. The graph describes who communicates with whom and under what relational context. The edge describes the flow being classified. The graph-based IDS literature therefore motivates an edge-

level representation retaining both endpoint structure and edge-specific content. These two architectures are also the ones re-trained on our own aggregated representation as baselines in Section 4.6. The next question is whether edge content has to remain numeric, or whether some of it can be represented semantically.

2.2 Language models over security text and network traffic

Cybersecurity language encoders provide one route to domain-specific semantics. Cy-SecBERT is a BERT-class encoder adapted to cybersecurity text [3]. Its relevance here is representational: a pretrained cybersecurity language encoder maps controlled security-related text into a dense embedding without requiring the graph encoder to interpret textual tokens itself.

A separate line of work treats traffic as a language-like sequence. BSTFNet applies a BERT-class model, ET-BERT, to serialised packet-level traffic and combines that representation with a spatiotemporal branch for encrypted-traffic classification [12]. It uses no graph encoder. The precedent that matters is the serialisation step itself: network observations that begin as structured traffic fields can be converted into a controlled token sequence and processed by a BERT-class encoder.

These two lines solve different problems. Domain adaptation addresses the vocabulary and semantics of cybersecurity text. Traffic serialisation creates text-like input from network measurements. Together they support a semantic branch that encodes a flow description as text rather than treating every field as a scalar magnitude. They do not, by themselves, determine how that semantic representation should interact with graph structure.

That interaction is the central fusion problem. Once a system holds one representation derived from topology and another derived from text, the design space is no longer only about which encoder to choose. It is about where the two modalities meet, and whether their relative influence is fixed across samples or allowed to change from one sample to the next.

2.3 Fusing structural and semantic representations

Fusion methods organise along two related axes: how tightly the modalities interact, and whether the fusion rule assigns the same modality weighting to every sample or adapts that weighting to the input. The second axis is the more important one here. Four recurring families cover the relevant design space.

The simplest family fixes the interface between the branches in advance. BertGCN is the clearest example: its final prediction linearly interpolates a GCN prediction and

a BERT prediction using a single global hyperparameter [16]. Every sample receives the same interpolation coefficient. Such designs keep the encoders modular, but the fusion rule never decides that one modality should matter more for one input and less for another.

A second family introduces input-dependent gates. The Gated Multimodal Unit uses multiplicative gates to determine how each modality contributes to the fused activation [1]. This is the canonical primitive for sample-adaptive modality weighting. A graph-and-text instance appears in the Dual-Stream Graph-Augmented Transformer for fake-news detection, where BERT runs in parallel with a GAT and Graph Transformer branch, and a trainable scoring function emits a scalar weight for each modality for each input [21]. Mechanically this is the closest analogue in the literature surveyed here to a fusion stage whose balance changes per sample.

A third family uses attention to let the modality representations interact directly. RAGFormer stacks the two branch embeddings as a short token sequence and applies self-attention over them with a residual connection [14]. CAST uses cross-attention between graph and text representations in materials prediction and motivates the text branch by arguing that graph conversion can discard global information which textual descriptions restore [13]. The semantic representation is therefore not a duplicate encoding of graph structure. BiGTex goes further and interleaves a two-way exchange: structural information enters the language model as a soft prompt, and text returns to the graph side through cross-attention with the query supplied from the graph side [4]. GMLM runs the exchange in both directions at once: the language embeddings attend to the graph embeddings and the graph embeddings attend to the language embeddings, and the two attended results are concatenated before classification, a design its authors set explicitly against simple concatenation [30]. Attention-based interaction differs from global interpolation because its interaction weights are derived from the representations of the current input.

The fourth family removes a distinct late-fusion stage altogether by interleaving structural and semantic processing throughout the encoder. DET uses exactly the terms “structural encoder” and “semantic encoder” and argues against late fusion of separate encoders, preferring interleaved interaction [10]. GL-Fusion sits at the opposite end of the coupling spectrum from independent towers: message passing is folded into every transformer layer, leaving no separate fusion module [36]. These designs mix the modalities repeatedly instead of preserving two independent representations until classification.

The taxonomy separates two questions that are often collapsed. One is where graph and text meet: after separate encoding, inside a dedicated attention mechanism, or repeatedly throughout the encoder. The other is whether the relative influence of the

modalities is globally fixed or varies with the sample. The fusion stage built in this report belongs to the second family. A gate reads both projected representations and emits a pair of weights that sum to one, so the weighting between structure and semantics is decided separately for every edge rather than by a single coefficient shared across the dataset. The second distinction becomes sharper on moving from general graph-text systems to intrusion detection, where the available GNN-language combinations tend to assign the two components sequential roles rather than fusing their representations.

2.4 GNN and LLM combinations in intrusion detection

Within the intrusion-detection works surveyed here, graph models and language models are combined sequentially rather than fused as two representations of the same edge-level decision. XG-NID is the clearest example. Its heterogeneous GNN performs the intrusion classification, and an LLM is used only after prediction, to produce human-readable explanations [8]. The system’s “dual modality” refers to flow-level and packet-level data inside the graph pipeline, not to graph and text embeddings fused for classification.

A poster by Zhan et al. [37], which describes its aim as enhancing GNN “robustness” for network intrusion detection, reverses the order. An LLM agent analyses textual descriptions of node interactions and flags, removes, or reweights suspicious nodes and edges before the GNN processes the graph. The language component acts as a pre-processing filter: its output changes the graph presented to the GNN, but the two components are never combined as concurrent representations at the classifier.

Other intrusion-detection systems occupy adjacent points in the design space without forming a GNN-language pair at all. GCN-2-Former combines a GCN with a temporal transformer and fuses the two by global average pooling and concatenation [38]. The second tower is a temporal transformer, not a language model. CPS-IDS uses gated fusion between a DeBERTa payload-semantic branch and a statistical branch [35], so it has both a language-based semantic component and an explicit fusion mechanism, but no graph encoder.

The distinction is functional rather than terminological. In this literature the language model either explains a GNN output after classification or alters graph inputs before message passing. The parallel-fusion examples occupy nearby configurations but either replace the language model with a non-language temporal transformer or replace the graph encoder with a statistical branch. Representation-level fusion and graph-language feedback therefore have to be examined in the broader graph-learning literature rather than inferred from the intrusion-detection examples alone.

2.5 Iterative GNN–LLM feedback

The broader GNN–LLM literature provides four relevant feedback approaches.

DAS is the closest prior work to the feedback mechanism used in this report. It implements an explicit closed feedback loop between a frozen GNN and an LLM: GNN predictions provide implicit supervision that steers semantic refinement, and the revised semantics then re-enter the same graph learner [32]. DAS also writes each node’s structural statistics out as a sentence, using degree, betweenness, closeness, clustering coefficient and square clustering coefficient, and gives a Majorization–Minimization argument, the technique of repeatedly minimising an easier upper bound in place of a hard objective, under which repeated refinement never increases the surrogate objective it defines for the task. Its setting is node-level, and it is evaluated on graphs that carry node text and on graphs that do not, where that structural sentence is what supplies the missing text. Its reported gains concentrate on the structure-dominated graphs.

LOGIN already couples uncertainty selection to language-model consultation. It uses MC-dropout variance, the spread of predictions obtained by running the network several times with dropout left switched on, to identify uncertain nodes. It then prompts the LLM with the node’s own text and a description of its two-hop neighbourhood, and receives a predicted label together with a free-text explanation [26]. Incorporation depends on correctness: if the LLM’s label matches the ground-truth label, the explanation is appended to the node text and the augmented text is re-embedded. If the label is wrong, edges in the node’s one-hop neighbourhood are pruned. The LLM remains outside autograd and edits the learning inputs between training rounds.

GLANCE makes the decision to consult the language model itself learnable. A lightweight router decides per node whether querying is worthwhile from inexpensive node features, and an advantage-based objective trains that router on the outcome of querying versus not querying, because the LLM call is non-differentiable [19]. Its gains concentrate on heterophilous and structurally ambiguous nodes. This addresses the same allocation problem as uncertainty-based consultation, directing language-model effort at selected graph elements rather than applying it uniformly, with a learned criterion in place of a fixed uncertainty rule.

RoGRAD places iterative refinement, rather than static one-shot signal injection, at the centre of its formulation [34]. It studies that refinement under graph deficiencies including noise, sparsity, and label scarcity. Its abstract claims priority for moving language-model augmentation of graphs from static injection to an iterative refinement paradigm.

2.6 Positioning

The contribution is an edge-level intrusion-detection instantiation, and a measurement question about how such a feedback channel is used. The comparison has to be made at the level of configuration, not of individual mechanisms.

The four feedback anchors operate at node level, whereas the present setting treats a network flow as an edge and makes the classification decision there. Their semantic consultants are prompted generative LLMs. Here the semantic branch is a frozen sentence encoder, and the consultant reading its embeddings is a classification head trained on them. An earlier configuration scored those embeddings against class prototypes instead, and both are reported. Their feedback actions refine semantic inputs or edit graph inputs. Here the accepted semantic signal modifies the edge representations consumed by the next graph update. These are differences of instantiation, not claims that the underlying ideas originated here [19, 26, 32, 34].

Two signals travel in opposite directions in the resulting loop. The graph model’s per-edge uncertainty determines *which* edges are routed to the semantic branch, and that selection is recomputed as predictions shift. The semantic branch returns a per-edge judgement that re-enters the graph model’s edge representations. This is the routing-level bidirectionality defined in Section 1, and the content-level limit stated there applies here as well: the consultant’s verdict on an edge depends on that edge’s text alone.

The final classification stage adds a further dimension to the configuration. Structural and semantic edge representations are combined through gated fusion with sample-dependent modality weighting, rather than assigning the semantic component a pre-processing or post-hoc role. Sample-adaptive fusion is itself established outside this setting: gated multimodal units provide input-dependent modality control, and the Dual-Stream Graph-Augmented Transformer assigns per-input scalar modality weights to parallel text and graph representations [1, 21]. Representation-level graph–text interaction is likewise established, through self-attention, cross-attention, and interleaved exchange [4, 13, 14]. None of those mechanisms is a contribution claim here.

Within the limits of the literature search represented by this survey, we are not aware of prior work combining all three of the following in one system: bidirectional iterative coupling between a graph model and a semantic branch, sample-adaptive gated fusion for the final classification, and edge-level network intrusion detection. This is a statement about the configuration space our search covered. It does not imply a performance advantage, and it does not change the prior-art status of any individual component.

That positioning leaves a narrower research question. Existing feedback work asks whether a language-model-driven refinement process improves the graph-learning task, with the model revising semantics, modifying inputs, or selectively augmenting graph elements [19, 26, 32, 34]. The present work instead instantiates that loop in network-flow intrusion detection with a deliberately more constrained consultant, and asks a diagnostic question: whether such a consultant exploits a feedback channel that has the capacity to influence subsequent graph updates. That question is underexplored in the work surveyed here. No claim about its answer belongs in this section.

The contribution is best described as instantiation plus measurement. The system instantiates bidirectional feedback and sample-adaptive structural–semantic fusion for edge-level network intrusion detection, then measures whether the constrained semantic consultant makes use of the channel it is given. It does not rest on inventing GNN–LLM feedback loops, iterative semantic refinement, natural-language descriptions of graph statistics, or selective semantic consultation. Those mechanisms have clear precedents. The point of this configuration is to make their interaction measurable in an edge-level network-IDS setting.

3 Method

This section first describes the datasets, graph construction and feature encoding. It then explains the structural and semantic branches, how their representations are fused, and how semantic advice enters the feedback model. Finally, it presents the baseline settings and the common training and evaluation protocol.

3.1 Datasets and Common Processing Pipeline

We evaluate on NF-UNSW-NB15 and NF-ToN-IoT, the NetFlow-standardised variants of their respective intrusion-detection corpora [23, 28]. One pipeline processes both, and every architectural choice is shared between them. Only two quantities are permitted to differ, and both are selected on validation folds: the uncertainty threshold k and the feedback injection scale. This restriction is deliberate. A cross-dataset comparison in which the two ladders use different components measures the components, not the datasets.

GLASS converts flow records into a graph, derives node and edge attributes, trains an edge-level structural model, builds label-free semantic representations for the same edges, and combines the two modalities through adaptive fusion. Every classification decision is edge-level: a flow, or an aggregate of flows sharing a grouping key, is an edge, never a node. Figure 1 shows the whole pipeline and the four points at which it is scored.

3.2 Directed Multigraph Construction

Flows are aggregated into a directed multigraph keyed on

$$(\text{source IP, destination IP, attack type}). \quad (1)$$

Parallel edges therefore exist between an ordered IP pair whenever those flows carry different attack types. Nodes are the distinct IP addresses. The resulting graphs are small: NF-UNSW-NB15 yields 49 nodes and 656 edges, NF-ToN-IoT 1,501 nodes and 2,127 edges.

Each node carries ten centrality columns: betweenness, PageRank, degree, closeness, eigenvector, k -core, k -truss, global betweenness, global PageRank, and modularity vitality. For NF-UNSW-NB15, we compute these on an unweighted IP-to-IP graph with self-loops removed; NF-ToN-IoT supplies the columns in the source CSV. The following computation details describe our NF-UNSW-NB15 preprocessing. Betweenness (normalised), PageRank, closeness and eigenvector centrality are computed on the di-

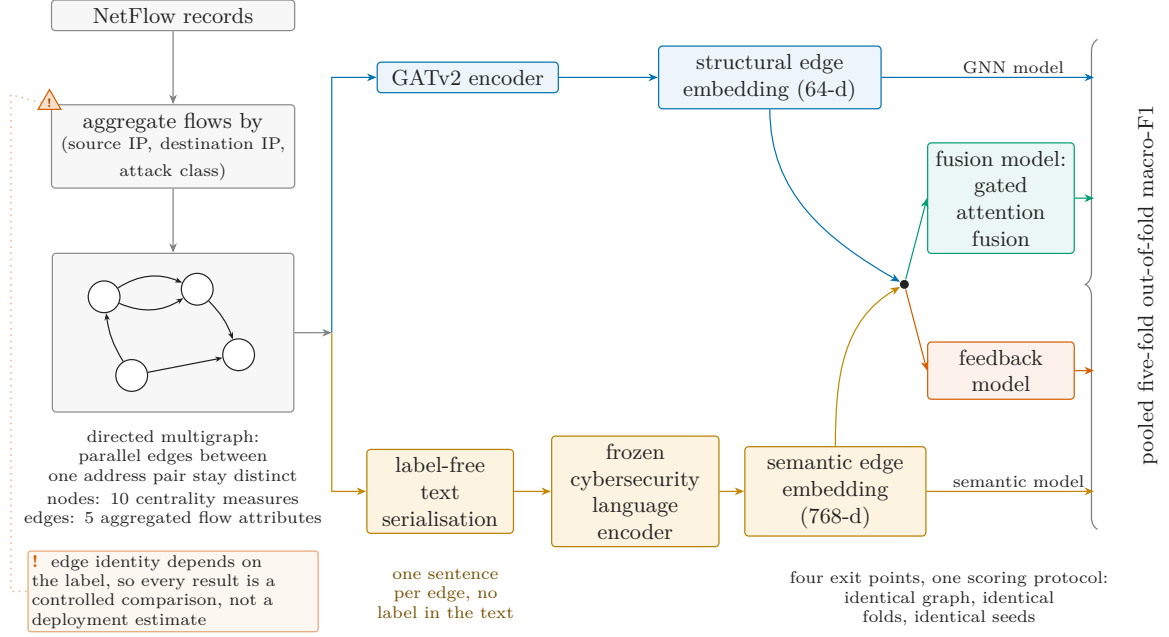


Figure 1: GLASS, from flow records to four scored rungs. Blue is the structural branch, orange the semantic branch, green the fusion stage and vermillion the feedback model. Every rung is an exit point from one shared graph and is scored by one protocol on one fold partition and one set of seeds. The marked aggregation step is the label-conditioned grouping key that bounds every claim in this report.

rected graph. Degree is *in-degree* centrality, and k -core and k -truss are computed on the undirected projection. Modularity vitality is the leave-one-node-out drop in modularity under a greedy modularity partition of the undirected projection. Eigenvector centrality and the truss decomposition fall back to constants when they fail to converge. A node’s feature vector is the mean of these values over all incident edges, taking the node in both its source and destination roles. In the processed graphs, each measure has the same value for an IP across its incident edges, apart from floating-point rounding. Averaging therefore recovers that value and reconciles the source-role and destination-role columns; it does not combine changing per-flow centralities.

One property of this feature set should be stated outright. On NF-UNSW-NB15 the “global” variants are populated from the same computation as their local counterparts, so `global_betweenness` and `global_pagerank` are exact duplicates of `betweenness` and `pagerank`. The ten columns span eight distinct measures, and one of those eight, `k_truss`, takes the value 2.0 at every node on this graph, so seven of the ten channels actually vary. On NF-ToN-IoT, whose centralities were computed upstream, the global variants differ from the local ones and `k_truss` varies. We report ten input channels because that is what the model receives, but the effective rank of the node features is lower on NF-UNSW-NB15.

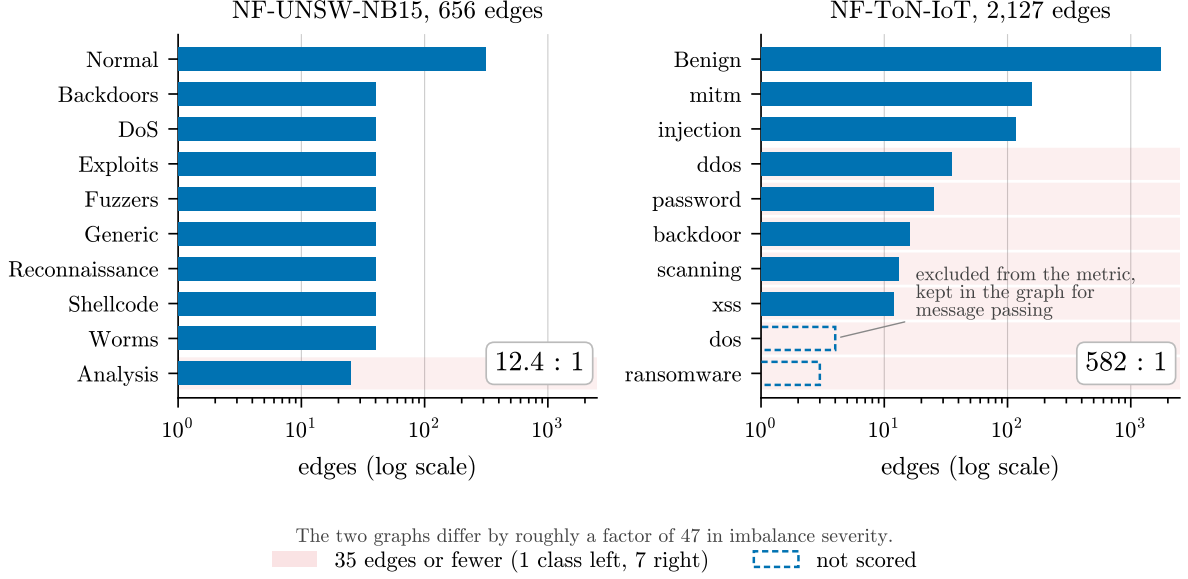


Figure 2: Class sizes on a logarithmic axis, largest first. NF-UNSW-NB15 has one class at or below 35 edges; NF-ToN-IoT has seven. The two smallest NF-ToN-IoT classes stay in the graph for message passing and are excluded from the metric. The two graphs differ by roughly a factor of 47 in the ratio of the largest class to the smallest class present in the graph, 582:1 against 12.4:1. Measured over the classes the metric scores, those ratios are 145.5:1 and 12.4:1.

Each edge carries five attributes. Three are numeric: the number of constituent flows (`flow_count`), their summed byte count (`total_bytes`), and their mean duration (`avg_duration`). Two are categorical: the modal protocol and the modal port, with ties broken toward the smaller value.

Attack type is part of the aggregation key, so the true label helps determine which flows form an edge. Two flows between the same hosts can become separate edges solely because their labels differ. A detector receiving unlabelled traffic could not construct those edges in that way. These results therefore compare representations and mechanisms under a controlled protocol, rather than estimate deployment performance [2, 31]. A deployment-valid version would need a label-independent grouping rule, for example endpoints and a time window, with a defined evaluation target when an aggregate contains mixed traffic. Features would need to use only information available at prediction time, and evaluation would need to respect traffic order [25]. These changes would alter the graph and task, so all models would have to be trained and evaluated again. Figure 2 gives the resulting class sizes on both graphs.

3.3 Edge Feature Encoding

The encoding separates quantities from identifiers. Columns 0–2 receive a $\log(1 + x)$ transform, then Z-scoring. Columns 3–4 are rewritten as contiguous vocabulary indices and consumed by learned embedding tables of width 4 (protocol) and 32 (port). Concatenation gives a $3 + 4 + 32 = 39$ -dimensional encoded edge vector. The vocabularies are small: 7 protocols and 169 ports on NF-UNSW-NB15, 4 and 235 on NF-ToN-IoT.

This corrects a specific defect in an earlier version of the pipeline, which passed protocol and port through as raw magnitudes. Port then entered the network at roughly $11,000\times$ the scale of every normalised column, and the ordering it imposed is meaningless: port 443 is not “more” than port 80. The embedding tables let the model represent protocol and port identity without inheriting the arithmetic of their indices.

The Z-score statistics and the two vocabularies are fitted once over all edges, before the cross-validation split. This is transductive, and we flag it explicitly. It is standard for a single fixed graph, and it transfers no label information, but it is not the same as fitting scalars inside each training fold.

3.4 Structural Branch

Each structural encoder is a two-layer GATv2 network [5]. The first layer uses eight heads of width $64/8 = 8$, concatenated to 64. The second uses a single head of width 64. Both layers receive the 39-dimensional encoded edge vector as an attention input and add no self-loops. Each layer applies, in order, the GATv2 convolution, batch normalisation, an ELU nonlinearity, and a residual connection. Dropout at $p = 0.2$ is applied before the residual in the first layer only. The first layer’s residual passes through a linear projection from the 10 input channels to 64, and the second’s is an identity. The final node embedding is 64-dimensional.

We use GATv2 rather than GAT because of the static-attention limitation [5]: in GAT [33] the ranking a node induces over its neighbours can be independent of the query node, so attention degenerates into a global neighbour scoring. GATv2 applies its attention MLP after the concatenation of both endpoints, which restores query-dependent ranking. On a graph with 49 nodes and heavy parallel-edge structure, that distinction is not cosmetic.

Three encoders run in parallel: *temporal*, *content* and *behavioural*. They are architecturally identical and differ only in a fixed diagonal reweighting of the encoded edge vector, given in Table 1. These weights were fixed a priori and never swept. No ablation was run to establish whether the three weighted branches improve on a single unweighted branch. They are source-code constants, not selected values. The protocol

Table 1: Per-variant edge focus weights. Applied as a fixed diagonal scaling of the encoded edge vector. The protocol and port weights are broadcast across their embedding blocks.

Variant	flow_count	total_bytes	avg_duration	protocol	port
temporal	1.25	0.75	1.50	0.50	0.25
content	0.50	1.50	0.50	1.00	1.25
behavioural	1.25	0.75	1.00	0.75	1.50

weight is broadcast across all 4 protocol-embedding dimensions and the port weight across all 32, so each variant keeps the same relative emphasis it would have had on the five raw columns. The reweighted edge vector is used both inside the attention computation and in the edge representation that follows, so a variant’s emphasis shapes its message passing as well as its readout.

Within a variant, the representation of a directed edge $e = (u, v)$ concatenates the two endpoint embeddings, their elementwise product, their elementwise absolute difference, and the variant’s reweighted edge vector $\tilde{\mathbf{x}}_e$:

$$\mathbf{z}_e = [\mathbf{h}_u, \mathbf{h}_v, \mathbf{h}_u \odot \mathbf{h}_v, |\mathbf{h}_u - \mathbf{h}_v|, \tilde{\mathbf{x}}_e] \in \mathbb{R}^{4 \cdot 64 + 39} = \mathbb{R}^{295}. \quad (2)$$

Equation (2) retains both endpoint interactions and edge attributes. Endpoint concatenation alone cannot separate parallel edges, and 58.7% of NF-UNSW-NB15 edges share an IP pair with at least one other edge. The elementwise terms are symmetric functions of the endpoints and do not fix this on their own, but $\tilde{\mathbf{x}}_e$ does, by carrying the edge’s own features into the classifier.

Each variant projects $\mathbf{z}_e$ to 64 dimensions through a linear layer and ReLU. The three projections are concatenated and passed through a linear layer to 64, ReLU, and dropout, yielding the fused 64-dimensional edge embedding consumed by every downstream stage. A final linear layer emits class logits. Each variant additionally carries its own classification head, a two-layer MLP over $\mathbf{z}_e$ with a 64-unit hidden layer, ReLU and dropout, whose loss is added to the objective with weight 0.30. The auxiliary term keeps each variant directly supervised rather than letting the fusion layer absorb all gradient.

3.5 Semantic Branch

The second modality describes each edge in controlled natural language. The continuous edge quantities are first discretised into five rank-based quantile bands, computed globally across all edges, and the band label rather than the raw number enters the sentence.

Those band edges are fitted before the cross-validation split, so they are transductive in the same way as the edge-encoding statistics above, and for the same reason: one fixed graph. A template then renders endpoints, connection frequency, transferred volume, per-flow byte magnitude, duration band, protocol name, and a coarse port role (web, remote-access, well-known, registered, or ephemeral) into one sentence per edge, of the form: “*Observed traffic from source ... to destination ... with ... connection frequency, transferring ... as ..., ..., over ..., to a ... port ...*”

Sentences are encoded by CySecBERT [3], a BERT-class encoder [6] domain-adapted to cybersecurity text. It is frozen throughout. No gradient reaches it. Inputs are tokenised with padding and truncation at 128 tokens, and the final hidden states are mean-pooled with padding positions masked out, giving one 768-dimensional vector per edge. Mean pooling over token states is the pooling Reimers and Gurevych [27] report as the stronger of the two common choices for a sentence vector, and we did not compare it against the first-token alternative on this data. CySecBERT is a pretrained language model, and we use it strictly as a sentence encoder: it is not prompted, not asked to reason, and not generative. Where this report says *language model* it means that pretrained encoder, not a generative one.

We evaluate two ways of classifying the same frozen embeddings. The *prototype scorer* first estimates a whitening transform from the training-fold embeddings, then computes a mean embedding for each class using its training labels. It assigns an edge to the closest class prototype by cosine similarity. This is the semantic stage in the four-stage comparison. It uses labelled examples to estimate prototypes, but does not train a neural classification head or update CySecBERT. Its score measures the embeddings together with this particular readout, not the encoder in isolation. The fusion model uses the embeddings directly, without this scorer.

The *trained head* learns a classification function from those embeddings. Within each fold, a standard scaler is fitted on the training embeddings and a 768–128–128– C multilayer perceptron predicts the classes, where C is ten on both graphs. The projection uses batch normalisation and ReLU; the hidden layer uses ReLU and dropout 0.2. Training uses the class-weighted focal loss, AdamW optimiser and early-stopping settings in Section 3.9. CySecBERT stays frozen. The head is evaluated on held-out edges and also supplies the feedback model’s semantic advice. We report its standalone performance as a component comparison, without adding a stage to the four-stage ladder.

The prototype gives a simple reference for reading the embeddings. We introduced the trained head after the prototype-feedback treatments in Section 4.4 failed to establish the intended gain. Training a head allows the decision boundaries to adapt to the labelled task instead of classifying solely by distance to a class average. The results

support this choice as a stronger semantic consultant, but do not establish that the surrounding feedback mechanism improves on it. Each fold’s stored head predictions cover all edges: training edges receive in-fold advice, while test edges receive advice from a head that has not trained on their labels. A cross-fitted variant replaces training advice with inner held-out predictions; its diagnostic results are reported separately in Appendix A.5.

Label freedom is enforced in code, not by convention. Before any sentence reaches the encoder, an assertion scans every generated sentence for every attack term present in the dataset and raises on the first match, naming the offending index and sentence. The pipeline halts. This is what makes the semantic branch an independent view of the edge rather than a relabelling of it.

3.6 Adaptive Gated Attention Fusion

The GLASS fusion model projects each modality into a shared 128-dimensional space through a linear layer, batch normalisation and ReLU, so 64 to 128 for the structural embedding $\mathbf{g}_e$ and 768 to 128 for the semantic one $\mathbf{l}_e$. A gate then decides the balance between them, not as one weight per edge but as one weight per dimension. It reads both projections together with their elementwise absolute difference and product, and emits a 128-dimensional vector of independent sigmoid weights $\boldsymbol{\gamma}_e \in (0, 1)^{128}$ that combines the two views channel by channel:

$$\mathbf{f}_e = \boldsymbol{\gamma}_e \odot \mathbf{g}_e + (\mathbf{1} - \boldsymbol{\gamma}_e) \odot \mathbf{l}_e, \quad \boldsymbol{\gamma}_e = \sigma(W[\mathbf{g}_e, \mathbf{l}_e, |\mathbf{g}_e - \mathbf{l}_e|, \mathbf{g}_e \odot \mathbf{l}_e]). \quad (3)$$

Equation (3) is the gated multimodal unit’s feature-wise form [1] rather than a single scalar blend: the weights are per-edge *and* per-dimension, so an edge can take some channels from structure and others from semantics instead of choosing one modality wholesale. Where this report quotes a gate value it is $\bar{\gamma}_e$, the mean over the 128 channels, read as the share given to the structural branch.

A second, feature-wise attention step follows. A linear map over $\mathbf{f}_e$ is softmaxed across its own 128 dimensions and multiplied back into it, reweighting the fused vector’s channels against one another before classification. A two-layer MLP over the result, with a 128-unit hidden layer, ReLU and dropout 0.2, produces class logits.

The objective adds one term to the focal loss of Section 3.9. The mean binary entropy of the gate on training edges is subtracted with weight 0.01, which rewards gates that stay away from 0 and 1 and so discourages the module from collapsing onto one modality before it has a reason to. Both inputs are standardised before entering the

module. The scalars are held in the trainer and persisted separately, so the module’s `state_dict` stays free of fitted statistics.

The structural embeddings supplied to this stage require particular care. The fusion model consumes *per-fold out-of-fold* structural embeddings only, never embeddings from a GNN fitted on all edges. Such a model has already seen the test edges’ labels, and those labels would reach the fusion classifier through its own input features, which is a leak no split downstream of it could detect. Earlier runs in this project used exactly that, and the reproduction entry point now enforces the out-of-fold rule.

The guarantee has a limit. For each held-out fold, the embeddings of that fold’s *test* edges come from a GNN that never saw their labels. The embeddings of its *training* edges come from the other folds’ GNNs, each of which was fitted on a partition that includes some of the held-out fold, so the fusion classifier is fitted on features that are one step removed from the test labels. This is the standard out-of-fold stacking construction and is what the published numbers use. A fully nested construction, with an inner split inside every training fold, would remove even that step and was not run.

3.7 Uncertainty-Guided Semantic Feedback

The feedback stage has two routes for semantic information. The injection route selects uncertain edges, reads the consultant’s class predictions and adds a learned projection of that advice to their edge features before another graph pass. The output-fusion route combines semantic and structural predictions at the final classifier. The isolation experiment disables output fusion so that injection can be measured separately. Figure 3 gives the stage as it runs.

Selection. For each edge we compute the Shannon entropy, in nats, of the GNN’s predictive distribution, and flag the edges above the $(1 - k/100)$ quantile. The quantile is recomputed from the current predictive distribution at every pass, so the flagged set is always exactly the top $k\%$ of that pass and its membership can change between passes. We select k on validation folds only, over $[15, 35]$ at three training seeds: $k = 29.0$ on NF-UNSW-NB15 and $k = 16.0$ on NF-ToN-IoT. The validation curves are flat, with every candidate inside the noise band of the mean, so these values are the argmax of a flat curve rather than a tuned optimum, and we say so wherever they appear.

Consultation. The feedback model consults the per-fold trained head described in the semantic branch. For every flagged edge it reads that fold’s stored head logits, a C -dimensional class judgement computed from the edge’s sentence alone. The judgement

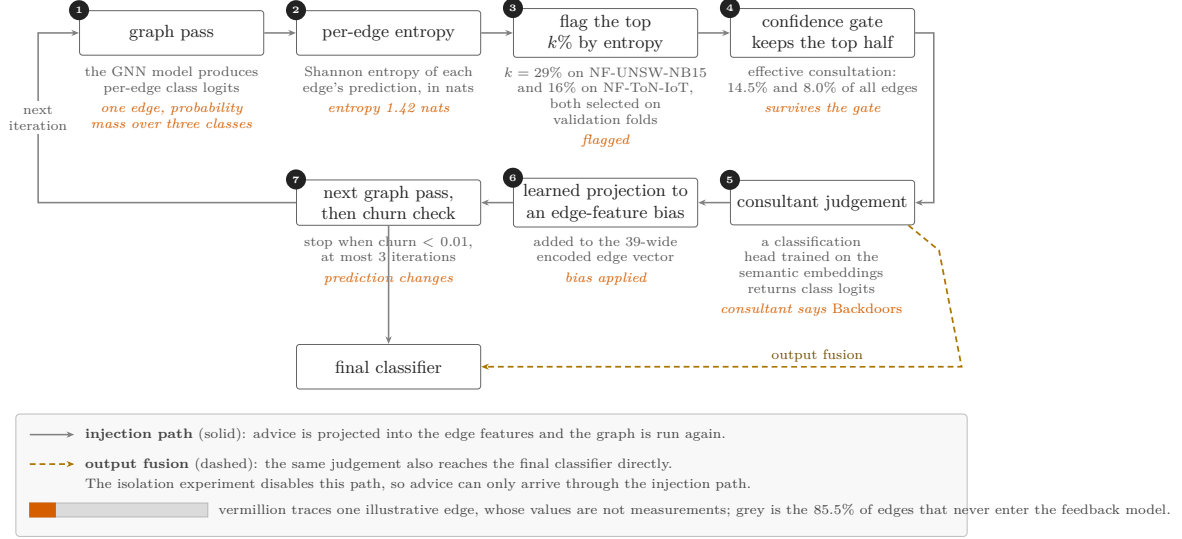


Figure 3: One pass of the feedback model. The solid path is the injection mechanism under test: the consultant’s judgement is projected into the 39-wide encoded edge vector and the graph is run again. The dashed path is output fusion, which reaches the final classifier without passing through the graph, and which the isolation experiment of Section 4.3 disables. The percentages are the selected entropy thresholds and the effective consultation rates they produce. The vermilion trace follows one illustrative edge; its entropy, gate outcome and consultant verdict are not measurements.

does not depend on the graph model’s current belief about the edge, so the coupling is bidirectional in routing but not in content.

The embedding-only alternative, used by the semantic model and by the prototype-consultant configuration of the feedback model, is a whitened prototype scorer. The fusion model does not use it; as Section 3.6 states, it consumes the standardised 768-dimensional embedding directly. Within each fold we centre the training-fold embeddings, fit a ZCA whitener by eigendecomposition of their covariance with a floor on small eigenvalues (necessary because the training fold has fewer rows than the 768 embedding dimensions), and take the class-wise mean in the whitened space as that class’s prototype. An edge is whitened with the same fold-local transform and scored by cosine similarity against the prototypes. Whitening is not incidental: an earlier L2-normalise-average-renormalise scheme collapsed per-class spread until random and real advice became indistinguishable.

Both consultant configurations use the same graph, frozen semantic encoder, folds and feedback training seeds. The original configuration uses prototypes; the final one uses the trained head. Their injection scales are unchanged, but the selected consultation percentiles differ: 31 versus 29 on NF-UNSW-NB15 and 25 versus 16 on NF-ToN-IoT. Consequently, their score difference measures a configuration change, not a controlled change of consultant alone. The stored trained-head advice was generated at

seed 42 and reused for every feedback training seed, so feedback’s reported seed spread does not include variation from retraining the consultant.

Gating. Being uncertain in the structural view does not make an edge a good candidate for semantic advice. Among the flagged edges we keep only the most confident half by the consultant’s own maximum class probability, a semantic confidence fraction of 0.50. The effective feedback rate is thus 14.5% of edges on NF-UNSW-NB15 and 8.0% on NF-ToN-IoT.

Injection. Accepted advice is injected into the *edge representation* consumed by the next graph update. The consultant’s class logits are projected linearly to the 39-dimensional edge width, scaled by a learned strength, written into a full-graph bias tensor that is exactly zero on unflagged edges, and added to the reweighted edge vector:

$$\tilde{\mathbf{x}}_e \leftarrow \tilde{\mathbf{x}}_e + s \cdot \mathbf{b}_e, \quad \mathbf{b}_e = \mathbf{0} \text{ for unflagged } e, \quad (4)$$

with injection scale $s = 2.0$ on NF-UNSW-NB15 and $s = 20.0$ on NF-ToN-IoT, both selected on validation folds from $\{0.5, 1, 2, 5, 10, 20\}$ at three seeds. As with k , the curves are flat, and on NF-ToN-IoT the selected value sits on the upper boundary of the swept range, which we record. Because $\tilde{\mathbf{x}}_e$ enters both attention and readout, the injected signal propagates structurally rather than only shifting a final logit. The scale is re-selected whenever the edge encoding or the injection path changes, since it is expressed in the units of that encoding.

Iteration. Predictions are recomputed, entropies re-derived, and the cycle repeats. We measure *churn*, the fraction of edges whose predicted class changed since the previous iteration, and stop when churn falls below 0.01 or after three iterations, whichever comes first.

Output fusion. The feedback model’s prediction is not the graph head’s logits after the last iteration. The refined edge embedding and the consultant’s logits are each projected to 128 dimensions and mixed by a per-edge gate $g_e \in (0, 1)$ that reads the entropy and maximum probability of both the graph head and the consultant, so it can route towards whichever view is confident on that edge. A two-layer classifier over the gated concatenation produces a correction, and the final logits are that correction plus the gated residual $(1 - g_e) \mathbf{z}_e^{\text{gnn}} + g_e \mathbf{z}_e^{\text{head}}$. This gives the classifier direct access to the consultant’s judgement as well as the refined graph representation; it does not

guarantee a minimum score. This stage has the same projection width as the fusion model, so the feedback model and the fusion model differ in where semantics enter (inside the graph update, or only at the output) and not in fusion capacity. Switching the stage off, together with the injection, yields the `head_only` control used throughout Section 4: the same network, trained in the same code path, with no semantic input at all.

An alternative that was implemented and measured. We also built a variant that adds the projected semantic signal to GATv2’s unnormalised attention logits, before the per-edge softmax, injecting into the attention mechanism rather than the edge representation. It is zero-initialised, so an untrained bias reproduces stock GATv2 exactly. It is evaluated under the identical protocol and reported in Section 4 as a negative result. We retain it in the paper because the two variants isolate *where* in the network a semantic signal must land to have any effect at all, a question the positive result alone does not answer.

3.8 Baseline Protocol

We compare against re-implementations of two published graph-based intrusion detectors, E-GraphSAGE [17] and TE-G-SAGE [20], retrained on the same aggregated edges, the same labels and the same folds. Both are edge-level extensions of GraphSAGE’s sample-and-aggregate scheme [11]. Each baseline keeps its own published featurisation rather than ours. E-GraphSAGE standard-scales all five NetFlow columns, TE-G-SAGE applies log1p, standard scaling, correlation pruning at 0.995, and one-hot categoricals with rare-category folding.

Each baseline is run in three configurations. `as_published` uses the released architectural settings, with the protocol changes listed below. `refit` selects hyperparameters on our validation folds only. `plus_node_features` is deliberately non-faithful: it hands the baseline our ten centrality node features, isolating how much of any gap is representation rather than architecture. We label it as non-faithful wherever it appears. It is not E-GraphSAGE and it is not TE-G-SAGE.

Five deviations from the published specifications were unavoidable at this graph scale, and we record each one. (D1) TE-G-SAGE’s chronological 60/30/10 split cannot be reconstructed, because aggregation collapses per-flow timestamps. Our stratified folds are used for all models. A random partition removes the temporal ordering whose absence Pendlebury et al. [25] show inflates reported scores, so the baseline numbers here and our own are measured under the looser of the two protocols. (D2) TE-G-SAGE’s neighbour fanout of (25, 15) and batch size of 4096 both exceed the entire graph, so

training is full-neighbourhood and full-batch. (D3) Both released implementations train for a fixed epoch count with no validation split and no early stopping, 4,999 epochs for E-GraphSAGE and 20 for TE-G-SAGE, incompatible with a protocol that selects on validation folds. Baselines are given 300 epochs with patience 25 on validation macro-F1 and the best state restored. (D4) TE-G-SAGE’s `rare_min_freq` of 50 removes almost every port category at our scale. It is applied as published in `as_published` and swept in `refit`. (D5) E-GraphSAGE’s published Eq. 4 aggregates edge features alone, but the authors’ released code builds the message as $W_{\text{msg}}([\mathbf{h}_u || \mathbf{e}_{uv}])$. We follow the code, since the published numbers came from it.

Table 2 lists the settings stored with the reported runs. The refit values come from validation-only searches. Adding our node features keeps the refit settings and changes only the node inputs.

Table 2: Baseline hyperparameters from the saved run configurations. All use Adam, inverse-frequency class-weighted cross-entropy, a maximum of 300 epochs and validation patience 25. Width is the hidden dimension; decay is weight decay. Rare min. is the minimum category frequency for TE-G-SAGE, whose edge MLP width is 128. Published and refit E-GraphSAGE initialise nodes with ones; TE-G-SAGE uses a learned constant. The + features ablations replace that initialisation with our ten node features. Published refers to the architectural settings, subject to deviations D1–D5 in the text.

Dataset	Variant	Layers	Width	Dropout	Learning rate	Decay	Rare min.
<i>E-GraphSAGE</i>							
UNSW	published	2	128	0.2	0.001	0	–
UNSW	refit	1	64	0.2	0.0005	0	–
UNSW	+ features	1	64	0.2	0.0005	0	–
ToN	published	2	128	0.2	0.001	0	–
ToN	refit	2	128	0.2	0.001	0	–
ToN	+ features	2	128	0.2	0.001	0	–
<i>TE-G-SAGE</i>							
UNSW	published	2	128	0.3	0.0003	0.0001	50
UNSW	refit	2	32	0.3	0.001	0.0001	2
UNSW	+ features	2	32	0.3	0.001	0.0001	2
ToN	published	2	128	0.3	0.0003	0.0001	50
ToN	refit	1	32	0.3	0.001	0.0001	2
ToN	+ features	1	32	0.3	0.001	0.0001	2

Two asymmetries remain and should be weighed when reading the comparison. Our rungs train under a focal objective with patience 40, whereas the baselines use class-weighted cross-entropy with patience 25, each keeping its own published loss. And E-GraphSAGE classifies an edge from $[\mathbf{h}_u || \mathbf{h}_v]$ alone, so parallel edges between one IP pair are mathematically indistinguishable to it. That covers 58.7% of NF-UNSW-NB15 edges and 7.9% of NF-ToN-IoT edges. That is an interaction between its architecture

and our aggregated representation, not evidence that E-GraphSAGE is a weak detector, and we do not read it as one.

3.9 Evaluation and Training Protocol

We use stratified five-fold cross-validation over edges with partition seed 42, represented as boolean masks. Within each fold the held-out fifth is the test partition, and the remaining four-fifths are split 75/25 into training and validation with seed 42 + fold index, giving approximately 60/20/20. Test masks are used once, for prediction.

The primary metric is pooled out-of-fold macro-F1: each edge is predicted only while held out, and the five sets of out-of-fold predictions are pooled before a single macro-F1 is computed. Model selection uses validation folds exclusively. The uncertainty threshold k and the injection scale were selected per dataset by a validation sweep ($k \in [15, 35]$; scale $\in \{0.5, 1, 2, 5, 10, 20\}$) averaged across three seeds (42, 1, 2). The remaining architectural hyperparameters were fixed a priori and not swept. We note this as a limitation: k was selected on rotating validation folds rather than by nested cross-validation, so the final estimate is not fully unbiased with respect to that one choice.

Every rung is trained at three training seeds (42, 1, 2) with the fold partition held fixed at the seed-42 split, so all seeds score the same held-out edges and differ only in initialisation, dropout and data order. We report the mean and standard deviation of pooled out-of-fold macro-F1 over the three seeds. The standard deviation therefore measures training-seed variance at a fixed partition, not total uncertainty, since fold-partition variance is not measured. Pairwise differences between rungs use a two-level bootstrap [7] with 2,000 iterations: each iteration draws a training seed independently for each side and resamples edges with replacement, so the interval reflects both sources of variation. A seed-matched bootstrap (seed 42 against 42, 1 against 1, 2 against 2) is reported as a secondary check. Because the fold partition is fixed, that second procedure pairs the two sides on the edge sample and on the training run at once, which is why its intervals are the narrower of the two. The decision rule was fixed before the runs: a difference is called *separated* only if its two-level 95% interval excludes zero, and *not separated* otherwise; we do not describe point-estimate orderings as significant. Fixing the rule in advance and reporting intervals rather than orderings addresses two of the pitfalls Arp et al. [2] document in learning-based security evaluations, inappropriate performance claims and selection made after the numbers are seen. The semantic model is deterministic given the folds and has zero seed variance, so it enters comparisons as a fixed side. Comparisons against re-trained published baselines use the same procedure with three baseline seeds on the other side.

Training uses AdamW [18] with learning rate 10^{-3} and weight decay 5×10^{-4} , at most 300 epochs, early stopping with patience 40 on validation macro-F1 with the best state restored, and gradient clipping at norm 1.0. The objective is focal loss [15] with $\gamma = 2.0$ and α set to inverse-frequency class weights computed on the training fold, normalised to sum to the class count, and classes absent from a training fold receive weight zero. Focal loss down-weights already-correct examples, which matters when rare attack classes hold 12–35 edges against a benign majority. We do not additionally oversample: combining oversampling with these weights double-corrects the imbalance and collapses the fusion stage.

Every run exports `OMP_NUM_THREADS=1`. Graph-attention message passing sums neighbour contributions in thread-completion order, and floating-point addition is not associative, so thread count alone moves pooled macro-F1 by roughly 0.012 at a fixed seed, larger than several of the effects this paper reports. Semantic encoding is likewise pinned to a single device, because MPS and CPU produce slightly different embeddings and every downstream number inherits the difference.

Two NF-ToN-IoT classes are smaller than the five-fold minimum: `dos` with 4 edges and `ransomware` with 3. They remain in the graph and continue to participate in message passing, but are excluded from the reported metric, so NF-ToN-IoT macro-F1 is computed over 8 classes and NF-UNSW-NB15 over 10. The two are not comparable in absolute level, and we compare only the shape of the ladder across datasets, never the values.

Excluding a class from the metric is not the same as forbidding a model to predict it, and the two were not applied uniformly. The GNN model, the semantic model and the feedback model set the two excluded logit columns to negative infinity before taking an argmax, so none of them can spend a scored edge on a class the metric does not cover. The fusion model and the re-trained baselines were scored without that mask, so they can, and an edge lost that way is a miss the other three were unable to make. Section 4.6 gives the counts and bounds what the difference is worth. A second asymmetry sits beside it: on NF-ToN-IoT the GNN model selected its checkpoints on validation macro-F1 over all ten classes, while the feedback model and the baselines selected on the eight the metric reports. Both are fixed in the code, and the published numbers were produced before the fix and were not re-run.

4 Results

This section evaluates GLASS. It first summarises the evaluation protocol and compares the models. It then examines feedback ablations, the earlier prototype treatments and attention injection. Finally, it compares the re-trained baselines, examines individual classes and reports how the feedback settings were selected.

4.1 The protocol as it was run

All models were evaluated using pooled five-fold out-of-fold predictions at training seeds 42, 1 and 2, with the fold partition fixed at seed 42. We report macro-F1, accuracy and weighted F1. NF-UNSW-NB15 is scored over ten classes; NF-ToN-IoT is scored over eight, excluding `dos` and `ransomware` while retaining their edges for message passing. The semantic prototype model is deterministic for these folds and embeddings. All runs use `OMP_NUM_THREADS=1`; Appendix C gives the reproduction details.

Macro-F1 gives each evaluated class equal weight, so rare attacks remain visible even when benign traffic dominates. The reported standard deviations measure training variation at one fixed fold partition; they do not measure uncertainty from changing that partition. A difference is called *separated* only when its 95% two-level bootstrap interval excludes zero. This procedure resamples edges and independently draws a training seed for each model over 2,000 iterations. The seed-matched bootstrap is a secondary check, and the two-level result decides when they disagree. We compare model orderings across datasets, not their absolute scores, because the evaluated class sets differ. The graph-construction limitation in Section 3.2 applies to every result.

4.2 The model ladder

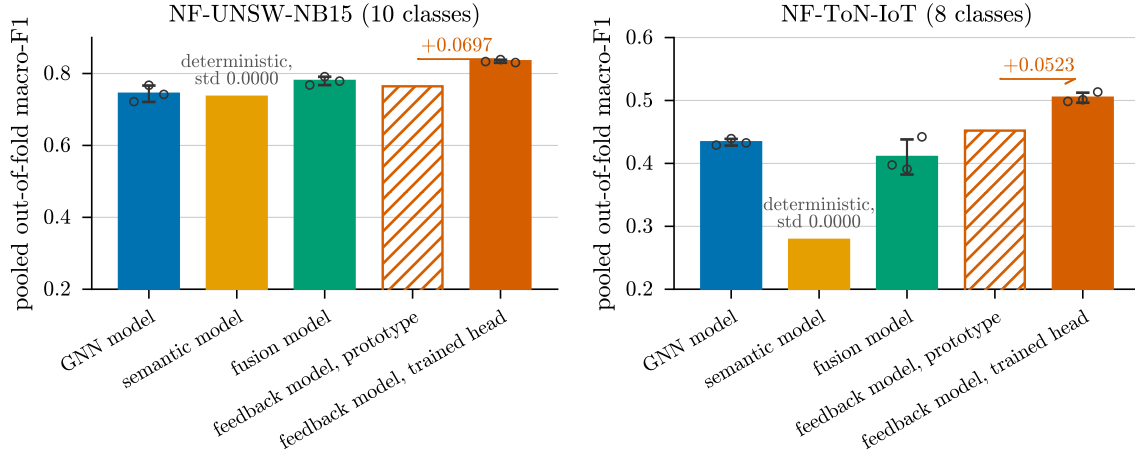
Table 3 and Figure 4 report the four-stage comparison. Table 4 also shows the trained head alone, alongside both feedback configurations, on all three metrics; Figure 5 plots the stage-level results. With the trained-head consultant, feedback reaches macro-F1 of 0.8341 ± 0.0042 on NF-UNSW-NB15 and 0.5044 ± 0.0080 on NF-ToN-IoT, compared with 0.7644 ± 0.0045 and 0.4521 ± 0.0114 with the prototype consultant. The head alone reaches 0.8374 ± 0.0048 and 0.5081 ± 0.0073 . Feedback is separated above the GNN, semantic and fusion stages on both datasets, but not above the head alone. Fusion is not separated from the GNN on either dataset, but it is separated above the prototype semantic model: the two-level differences are $+0.0443$ [$+0.0067$, $+0.0795$] on NF-UNSW-NB15 and $+0.1299$ [$+0.0704$, $+0.2054$] on NF-ToN-IoT. Table 5 gives both RQ2 comparisons and the feedback comparisons; Figure 6 plots the feedback and

fusion-versus-GNN intervals. Several evaluated classes contain only 12–40 edges, so these aggregate results should be read alongside the small-class caution in Section 4.7.

Fusion therefore improves on the prototype readout, but does not establish that combining the branches improves over both single-branch stages by separated margins. The trained head is also essential to interpreting the feedback result. It has the highest mean macro-F1 on both datasets, and adding feedback has not established an improvement over it. The feedback-minus-head intervals are $[-0.0228, +0.0128]$ on NF-UNSW-NB15 and $[-0.0342, +0.0299]$ on NF-ToN-IoT. Replacing the prototype consultant also changed the selected consultation percentile; the comparison cannot assign the gain to either change alone. The change between feedback configurations is separated on NF-UNSW-NB15, but not on NF-ToN-IoT under the primary bootstrap. Prototype feedback is not separated above either structural stage on either dataset; its only separated stage comparison is against the semantic model on NF-ToN-IoT. These findings justify examining the stronger consultant while testing the injection path separately in Section 4.3. Appendix B.4 gives the supporting intervals and explains why accuracy can hide weak rare-class performance.

Table 3: Pooled out-of-fold macro-F1 by rung, three training seeds at a fixed fold partition. The feedback model is reported under both consultants. The $\pm$ is training-seed variance only. Absolute levels are not comparable across datasets. The trained head alone has a higher mean macro-F1 than feedback on both datasets, without a separated difference; see the component comparison in Table 4.

Rung	NF-UNSW-NB15 (10 classes)			NF-ToN-IoT (8 classes)		
	mean $\pm$ std	seeds 42 / 1 / 2		mean $\pm$ std	seeds 42 / 1 / 2	
GNN model	0.7437 ± 0.0228	0.7219 / 0.7674 / 0.7418		0.4336 ± 0.0053	0.4290 / 0.4393 / 0.4326	
Semantic model	0.7353 ± 0.0000	0.7353 / 0.7353 / 0.7353		0.2785 ± 0.0000	0.2785 / 0.2785 / 0.2785	
Fusion model	0.7793 ± 0.0116	0.7680 / 0.7912 / 0.7788		0.4102 ± 0.0279	0.3975 / 0.3910 / 0.4422	
Feedback model, prototype consultant	0.7644 ± 0.0045	0.7642 / 0.7690 / 0.7601		0.4521 ± 0.0114	0.4414 / 0.4509 / 0.4641	
Feedback model, trained-head consultant	0.8341 ± 0.0042	0.8332 / 0.8387 / 0.8304		0.5044 ± 0.0080	0.4985 / 0.5012 / 0.5135	

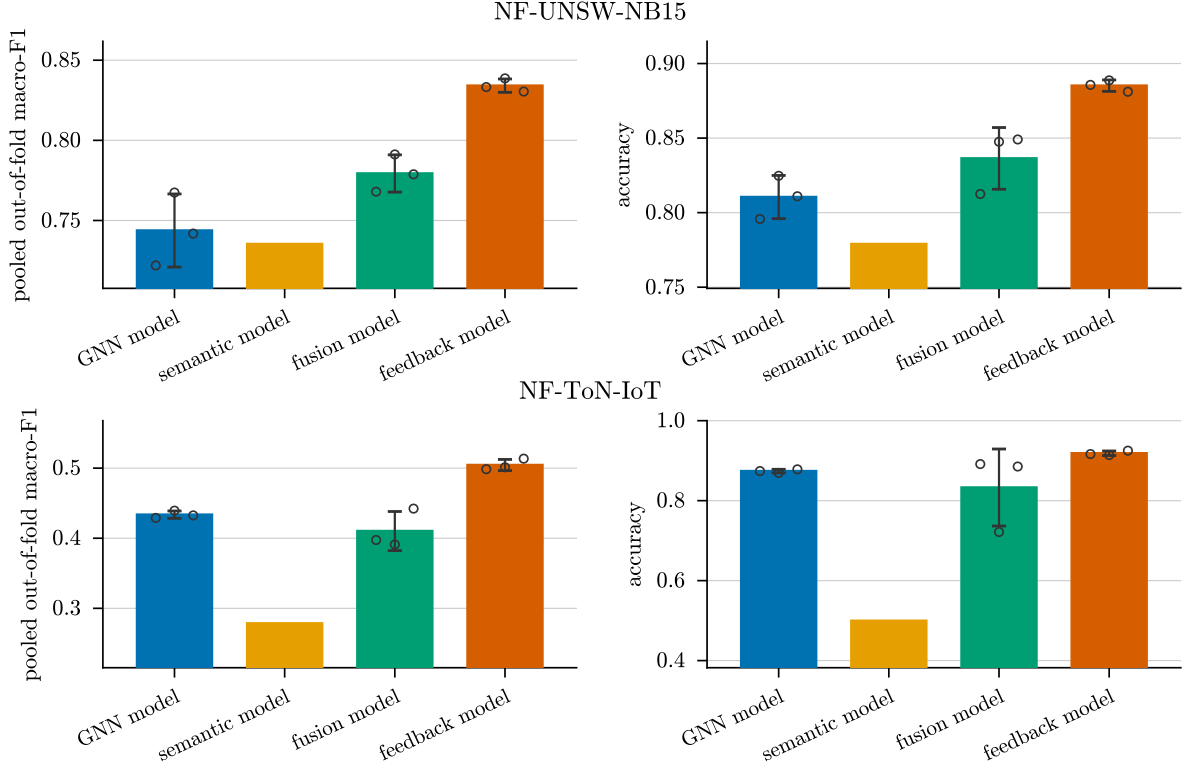


Error bars are training-seed variance at a fixed fold partition; fold-partition variance is not measured. Open circles are the three individual seeds. The consultant change is separated on NF-UNSW-NB15 and not on NF-ToN-IoT; the two feedback model versions also differ in their selected top-k (31 to 29, and 25 to 16). The two panels have independent y-axes.

Figure 4: The four rungs on both datasets, with the feedback model drawn under each of its two consultants. Error bars are one standard deviation over three training seeds at a fixed fold partition; fold-partition variance is not measured. Open circles are the three individual seeds. The semantic model is deterministic and carries no spread. The hatched bar is the superseded prototype-consultant feedback model; the arrow between the two feedback model bars carries an interval, given in the text. The two panels have independent y-axes that do not start at zero, and their absolute heights are not comparable.

Table 4: Model and component comparison, mean $\pm$ standard deviation over three training seeds at a fixed fold partition. The trained head is the feedback model’s semantic consultant, evaluated separately; it is not an additional stage of the four-stage ladder. Its mean macro-F1 is higher than feedback on both datasets, but their difference is not separated. Absolute scores should not be compared across datasets.

Model or component	macro-F1	accuracy	weighted F1
<i>NF-UNSW-NB15 (10 classes)</i>			
GNN model	0.7437 ± 0.0228	0.8105 ± 0.0145	0.8207 ± 0.0148
Semantic model (prototype)	0.7353 ± 0.0000	0.7790 ± 0.0000	0.7914 ± 0.0000
Fusion model	0.7793 ± 0.0116	0.8364 ± 0.0207	0.8398 ± 0.0157
Feedback, prototype consultant	0.7644 ± 0.0045	0.8237 ± 0.0049	0.8332 ± 0.0048
Feedback, trained-head consultant	0.8341 ± 0.0042	0.8852 ± 0.0038	0.8887 ± 0.0036
Trained head alone (component)	0.8374 ± 0.0048	0.8908 ± 0.0101	0.8933 ± 0.0083
<i>NF-ToN-IoT (8 classes)</i>			
GNN model	0.4336 ± 0.0053	0.8739 ± 0.0045	0.8949 ± 0.0040
Semantic model (prototype)	0.2785 ± 0.0000	0.4995 ± 0.0000	0.6071 ± 0.0000
Fusion model	0.4102 ± 0.0279	0.8329 ± 0.0963	0.8550 ± 0.0512
Feedback, prototype consultant	0.4521 ± 0.0114	0.8686 ± 0.0071	0.8914 ± 0.0049
Feedback, trained-head consultant	0.5044 ± 0.0080	0.9186 ± 0.0057	0.9223 ± 0.0048
Trained head alone (component)	0.5081 ± 0.0073	0.9164 ± 0.0027	0.9200 ± 0.0017

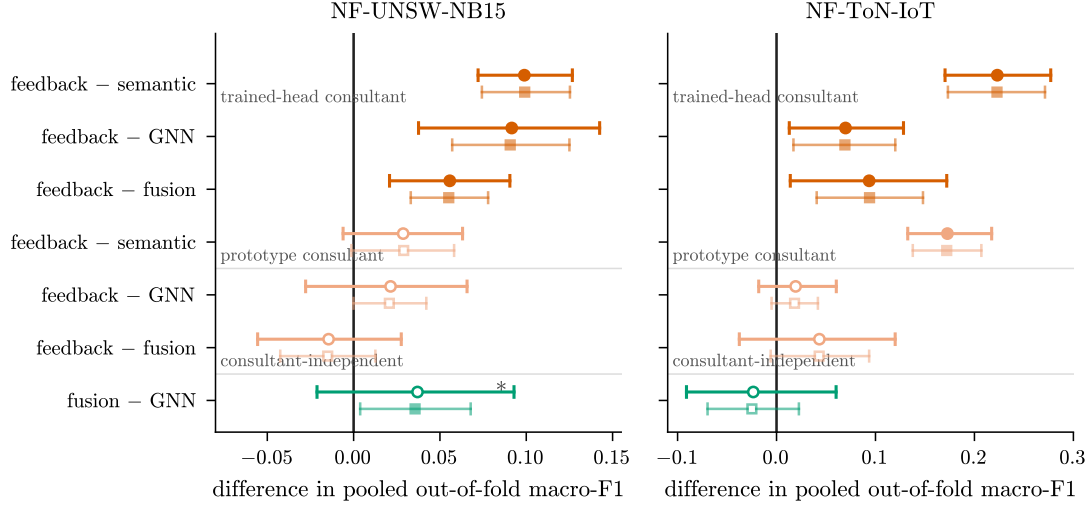


Left column macro-F1, right column accuracy, both on the rows the metric scores. Error bars are training-seed variance at a fixed fold partition; open circles are the three seeds. The semantic model is deterministic and has none. Every panel has its own y-axis. On NF-ToN-IoT the benign class holds 82% of scored edges, so accuracy stays high for every graph rung while macro-F1 separates them.

Figure 5: Macro-F1 and accuracy for the four rungs, both datasets. The feedback model is the canonical one, consulting the trained head. Error bars are one standard deviation over three training seeds at a fixed fold partition; fold-partition variance is not measured. Open circles are the three individual seeds. Every panel has its own y-axis and none starts at zero. The accuracy panels span a far narrower range than the macro-F1 panels, and on NF-ToN-IoT that is the benign majority showing through.

Table 5: Rung comparisons, both bootstrap procedures, 2,000 iterations. *Sep* marks an interval that excludes zero. *Stable* reports whether the sign of the per-seed difference is the same at all three seeds. Each comparison is given at both consultants, so a row’s verdict can be read against the consultant that produced it.

Dataset	Comparison	Consultant	two-level mean [95% CI]	sep	seed-matched mean [95% CI]	stable
UNSW	feedback – fusion	prototype	−0.0145 [−0.0556, +0.0276]	no	−0.0150 [−0.0425, +0.0126]	yes
	feedback – fusion	trained head	+0.0557 [+0.0208, +0.0905]	yes	+0.0551 [+0.0331, +0.0779]	yes
	feedback – GNN	prototype	+0.0214 [−0.0278, +0.0657]	no	+0.0207 [−0.0001, +0.0421]	yes
	feedback – GNN	trained head	+0.0916 [+0.0376, +0.1425]	yes	+0.0907 [+0.0571, +0.1250]	yes
	feedback – semantic	prototype	+0.0287 [−0.0061, +0.0631]	no	+0.0290 [−0.0013, +0.0582]	yes
	feedback – semantic	trained head	+0.0989 [+0.0721, +0.1267]	yes	+0.0991 [+0.0743, +0.1253]	yes
	fusion – GNN	either	+0.0370 [−0.0212, +0.0929]	no	+0.0356 [+0.0038, +0.0678]	yes
	fusion – semantic	either	+0.0443 [+0.0067, +0.0795]	yes	+0.0440 [+0.0194, +0.0698]	yes
ToN	feedback – fusion	prototype	+0.0433 [−0.0376, +0.1199]	no	+0.0431 [−0.0059, +0.0936]	yes
	feedback – fusion	trained head	+0.0936 [+0.0139, +0.1720]	yes	+0.0940 [+0.0406, +0.1480]	yes
	feedback – GNN	prototype	+0.0193 [−0.0180, +0.0604]	no	+0.0182 [−0.0050, +0.0418]	yes
	feedback – GNN	trained head	+0.0697 [+0.0129, +0.1283]	yes	+0.0691 [+0.0171, +0.1200]	yes
	feedback – semantic	prototype	+0.1727 [+0.1326, +0.2174]	yes	+0.1720 [+0.1377, +0.2070]	yes
	feedback – semantic	trained head	+0.2231 [+0.1703, +0.2771]	yes	+0.2228 [+0.1732, +0.2713]	yes
	fusion – GNN	either	−0.0235 [−0.0910, +0.0602]	no	−0.0249 [−0.0697, +0.0226]	no
	fusion – semantic	either	+0.1299 [+0.0704, +0.2054]	yes	+0.1289 [+0.0928, +0.1689]	yes



* On NF-UNSW-NB15, the fusion model minus the GNN model is separated under the seed-matched procedure and not once training-seed variance is admitted; the two-level result decides. Under the prototype consultant the feedback model against the semantic model is separated on NF-ToN-IoT and not on NF-UNSW-NB15.

—●— two-level (edges and training seed) —■— seed-matched (edges only) ○ interval includes zero

Figure 6: Feedback comparisons and fusion versus GNN as intervals, under each consultant. The additional fusion-versus-semantic intervals are reported in Table 5. A difference counts as separated only when the interval excludes zero, shown by a filled marker. The primary two-level procedure resamples edges and training seed; the seed-matched procedure resamples edges alone. With the trained head, feedback exceeds the GNN, semantic and fusion stages on both datasets; its comparison with the head alone is not separated (Table 4). Under the prototype consultant, only the comparison against the semantic model on NF-ToN-IoT excludes zero. Colours follow the rung the comparison is about, with the prototype group in a lighter tint.

4.3 Feedback ablations and the mechanism in isolation

Table 6 compares real advice, feedback off and random advice at each training seed. Real advice scores above both alternatives at every seed on both datasets. Table 7 and Figure 7 then test injection with output fusion disabled. The oracle, which deliberately receives the true label, improves over control by +0.0369 on NF-UNSW-NB15 and +0.1354 on NF-ToN-IoT, with both intervals excluding zero. The prototype consultant has no separated improvement. The trained-head consultant has no separated improvement on NF-UNSW-NB15 and a separated loss of -0.0378 on NF-ToN-IoT. This diagnostic used consultation percentiles of 31 and 25, respectively, rather than the final configuration’s 29 and 16; injection scales remained 2.0 and 20.0.

The first experiment measures access to semantic advice through the complete system, so it cannot identify injection as the source of the gain. The second shows that correct advice can help through the injection path, but neither realistic consultant produced a separated gain at the settings tested. The oracle is a diagnostic with deliberate label leakage, not a usable detector. Because the isolation settings differ from the final

configuration, this experiment does not directly measure injection’s contribution at the final settings. The separate head-alone comparison nevertheless shows no established advantage from the complete feedback model over its consultant. Here `head_only` means the graph-only control, not the trained semantic head. Prediction changes between passes show that the edge channel is active; activity alone does not establish better predictions. Appendix A.5 records the headroom calculation, cross-fitting check and churn evidence.

Table 6: Complete-system ablation, per training seed. The real-advice arm combines injection and semantic output fusion; the graph-only control has no semantic input. The interval is a paired bootstrap of real against `head_only`.

Dataset	Seed	real	<code>head_only</code>	random	real – <code>head_only</code> [95% CI]
UNSW	42	0.8332	0.7543	0.6385	+0.0792 [+0.0409, +0.1185]
	1	0.8387	0.7336	0.6511	+0.1054 [+0.0671, +0.1437]
	2	0.8304	0.7314	0.6378	+0.0994 [+0.0595, +0.1387]
ToN	42	0.4985	0.3809	0.4002	+0.1159 [+0.0596, +0.1751]
	1	0.5012	0.4404	0.3790	+0.0595 [+0.0074, +0.1148]
	2	0.5135	0.4374	0.4067	+0.0743 [+0.0223, +0.1280]

Table 7: Mechanism in isolation, output fusion disabled in every arm. Consultation percentiles are 31 (NF-UNSW-NB15) and 25 (NF-ToN-IoT), rather than the final 29 and 16; scales are 2.0 and 20.0. Means are over three seeds; intervals are paired over the fifteen (seed, fold) pairs. Captured share is the arm’s gain as a fraction of the oracle’s. The two rightmost rows are computed from the *mean* column of this table, headroom as oracle minus control and each share as that arm’s mean minus control over the same denominator; the interval column beside it reports a paired bootstrap mean over resampled (seed, fold) pairs, which is a different estimator. Dividing an interval column entry by the headroom therefore does not reproduce the share printed below it.

Arm	NF-UNSW-NB15		NF-ToN-IoT	
	mean	paired diff vs control [95% CI]	mean	paired diff vs control [95% CI]
control (<code>head_only</code>)	0.7398	–	0.4195	–
prototype, edge	0.7382	–0.0046 [–0.0273, +0.0158]	0.4325	+0.0099 [–0.0111, +0.0286]
trained head, edge	0.7441	+0.0007 [–0.0183, +0.0201]	0.3781	–0.0378 [–0.0588, –0.0167]
oracle, edge	0.7743	+0.0369 [+0.0097, +0.0618]	0.5286	+0.1354 [+0.0919, +0.1797]
oracle, attention	0.7382	–0.0007 [–0.0211, +0.0180]	0.4310	+0.0114 [–0.0083, +0.0307]
<i>Pooled headroom and captured share</i>				
oracle headroom		+0.0345		+0.1090
prototype share		–4.5%		+11.9%
trained head share		+12.5%		–38.0%

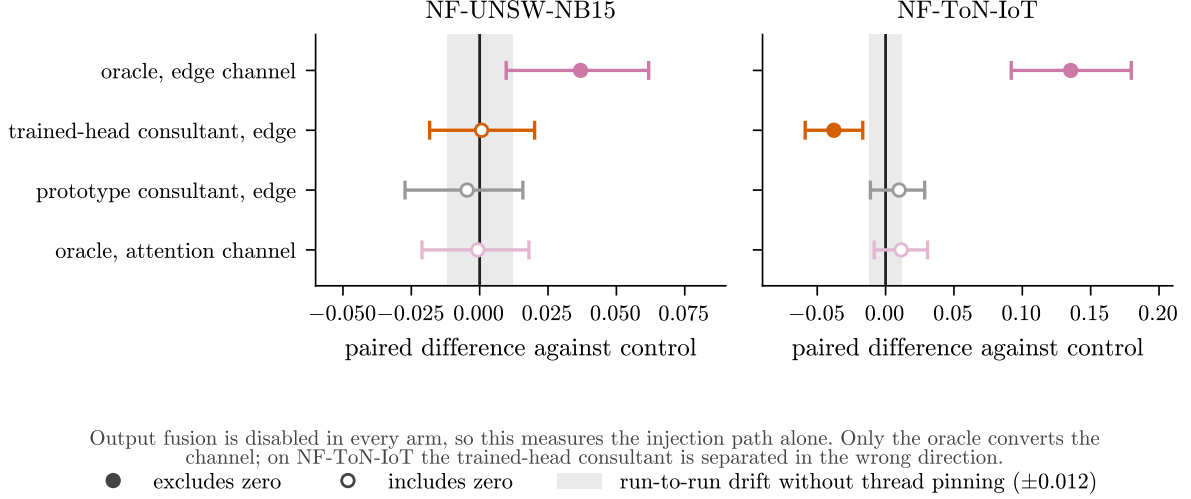


Figure 7: The injection path measured with output fusion disabled in every arm, so advice can reach the classifier only through the path under test. Filled markers mark intervals that exclude zero. The shaded band is the documented run-to-run drift without thread pinning, ± 0.012 macro-F1. The diagnostic uses consultation percentiles 31 and 25, not the final 29 and 16. Only the oracle improves through the edge channel on both datasets; on NF-ToN-IoT the trained-head consultant is separated in the wrong direction.

4.4 Five attempts on the embedding-only consultant

Five changes were tested with the prototype consultant: supervising the uncertainty selector, calibrating the consultant’s temperature, weighting advice by class reliability, changing the advice format, and cross-fitting the advice. None established the intended improvement. The first two were tested together over three seeds, with an additional condition that re-selected the feedback settings; none of these conditions separated feedback from fusion. The remaining treatments were single-seed diagnostics and do not support multi-seed conclusions. Table 14 and Appendix A give the settings and results. A separate seed-42 check on the uncertain edges found trained-head accuracy of 0.795 on NF-UNSW-NB15 and 0.648 on NF-ToN-IoT, compared with 0.647 and 0.284 for the prototype scorer (Appendix B.3).

These results motivated changing the consultant. The prototype assigns classes using average embeddings, whereas the trained head learns class boundaries from the same frozen embeddings and labelled training examples. Its stronger predictions on the consulted edges make it a better-supported choice for supplying semantic advice. The failed treatments explain why we investigated that choice; they do not prove that every possible prototype treatment would fail or that class imbalance caused the prototype’s weakness. The resulting feedback configuration has higher mean scores, but its consultation percentile also changed, and it does not have a separated advantage over the

trained head alone. We therefore retain both consultants in the results and treat the head-alone comparison as necessary evidence when assessing the feedback design.

4.5 Attention injection

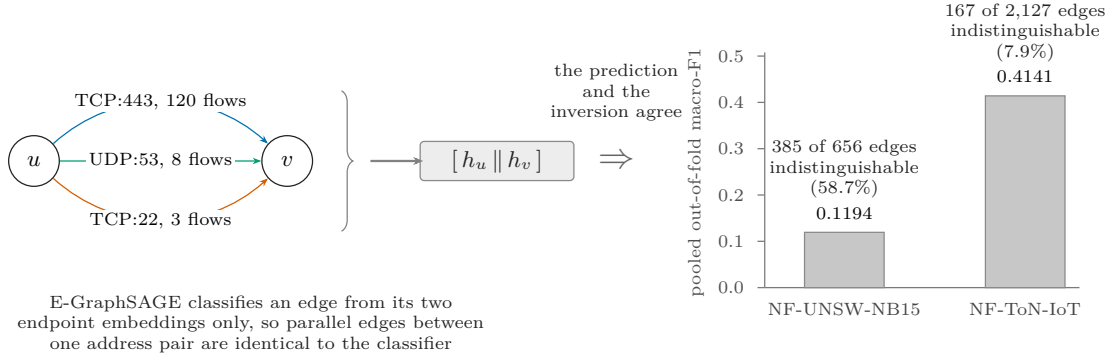
Attention injection produced prediction churn of exactly 0.0000 in every tested configuration, including an oracle supplied with the true label at ± 4 nats. Its bootstrap differences from control were -0.0007 $[-0.0211, +0.0180]$ on NF-UNSW-NB15 and $+0.0114$ $[-0.0083, +0.0307]$ on NF-ToN-IoT, neither separated from zero (Table 7). Unlike edge injection, this variant did not change the predicted classes between passes.

The graph structure helps explain this negative result. Attention changes node embeddings but cannot give different endpoint-derived representations to parallel edges between the same hosts; 58.7% of NF-UNSW-NB15 edges share an endpoint pair. On NF-ToN-IoT, 62.8% of edges terminate at a destination with only one incoming edge. For those destinations, the attention softmax is always 1.0, regardless of the injected bias. These constraints explain why the location of injection matters. They do not establish how much the original GNN benefits from message passing; that requires the separate comparison described in Section 5.7.

4.6 Re-trained published baselines

Tables 8 and 9 compare E-GraphSAGE and TE-G-SAGE re-trained on our aggregated representation. With its published settings, E-GraphSAGE reaches macro-F1 of 0.1194 on NF-UNSW-NB15 and 0.4141 on NF-ToN-IoT; TE-G-SAGE reaches 0.3841 and 0.1931. The tables also report validation-selected settings and variants using our node features. Trained-head feedback is separated above every faithful re-trained baseline, although it is not separated above its own semantic head. Prototype feedback is not separated from E-GraphSAGE on NF-ToN-IoT. On that dataset, neither the GNN nor the fusion stage is separated from E-GraphSAGE with its published settings. These comparisons use our protocol and do not compare against scores reported in the original papers.

Much of the baseline gap depends on representation and training choices. E-GraphSAGE classifies an edge from its endpoints alone, so it cannot distinguish parallel edges with different attributes. This affects 58.7% of NF-UNSW-NB15 edges and 7.9% of NF-ToN-IoT edges (Figure 8). Table 10 and Figure 9 separate the NF-UNSW-NB15 point-estimate increments associated with validation selection, node features and the remaining model change; those increments have no confidence intervals. The NF-ToN-IoT baselines and fusion stage also allowed predictions of excluded classes



Not a training-schedule artifact: at the authors' full 4,999 epochs on fold 0, validation macro-F1 plateaus near 0.11. Both bars are drawn in grey because neither is one of our rungs.

Figure 8: Why E-GraphSAGE collapses on one dataset and not the other. Its edge classifier reads only the concatenated endpoint embeddings, so parallel edges between one address pair are identical to it. The share of edges that are indistinguishable in this sense differs by roughly a factor of seven between the two graphs, and the scores invert with it. Both bars are grey because neither is one of our rungs.

in the reported runs. An optimistic correction bound remains below feedback except for one node-feature ablation at one seed, whose comparison was already not separated. Appendix B.5 gives this bound and the detailed comparisons. These qualifications prevent the results from being read as a general architectural advantage over the published detectors.

Table 8: Re-trained baselines, mean pooled out-of-fold macro-F1 over seeds 42, 1 and 2. *Faithful* marks a configuration that is the published architecture; + **node features** is a labelled ablation of ours and must not be tabulated under the paper's name.

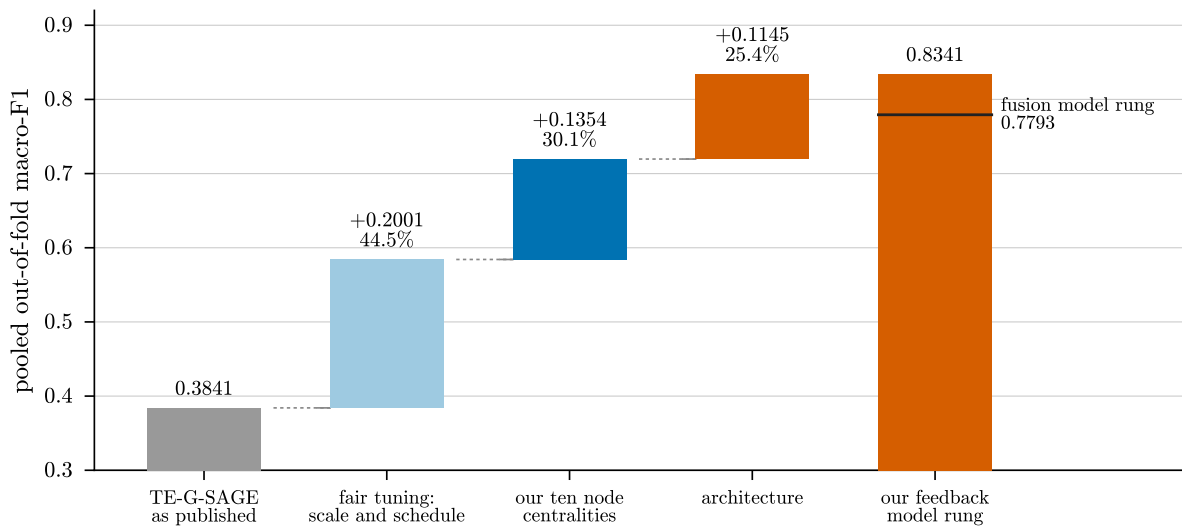
Architecture	Configuration	Faithful	NF-UNSW-NB15	NF-ToN-IoT
E-GraphSAGE	as published	yes	0.1194 ± 0.0065	0.4141 ± 0.0046
	refit	yes	0.1368 ± 0.0076	0.4141 ± 0.0046
	+ our node features	no	0.1173 ± 0.0065	0.4358 ± 0.0323
TE-G-SAGE	as published	yes	0.3841 ± 0.0078	0.1931 ± 0.0161
	refit	yes	0.5842 ± 0.0224	0.2345 ± 0.0101
	+ our node features	no	0.7196 ± 0.0094	0.3523 ± 0.0182

Table 9: The feedback model against each re-trained baseline, under both consultants, two-level bootstrap over edges and both seed sets. Positive favours our rung. *ns* marks an interval containing zero. Feedback is not separated from its own trained semantic head (Table 4).

Dataset	Baseline	Feedback model, prototype	Feedback model, trained head
UNSW	E-GraphSAGE, as published	+0.6428 [+0.5993, +0.6868]	+0.7130 [+0.6708, +0.7540]
	E-GraphSAGE, refit	+0.6257 [+0.5807, +0.6701]	+0.6960 [+0.6526, +0.7373]
	E-GraphSAGE + features	+0.6454 [+0.6005, +0.6901]	+0.7157 [+0.6731, +0.7558]
	TE-G-SAGE, as published	+0.3798 [+0.3257, +0.4336]	+0.4500 [+0.3924, +0.5042]
	TE-G-SAGE, refit	+0.1817 [+0.1216, +0.2454]	+0.2519 [+0.1914, +0.3169]
	TE-G-SAGE + features	+0.0453 [+0.0033, +0.0847]	+0.1155 [+0.0673, +0.1627]
ToN	E-GraphSAGE, as published	+0.0378 [−0.0098, +0.0857] <i>ns</i>	+0.0890 [+0.0312, +0.1480]
	E-GraphSAGE, refit	+0.0378 [−0.0098, +0.0857] <i>ns</i>	+0.0890 [+0.0312, +0.1480]
	E-GraphSAGE + features	+0.0156 [−0.0616, +0.0908] <i>ns</i>	+0.0667 [−0.0181, +0.1505] <i>ns</i>
	TE-G-SAGE, as published	+0.2572 [+0.2010, +0.3170]	+0.3083 [+0.2345, +0.3793]
	TE-G-SAGE, refit	+0.2161 [+0.1624, +0.2688]	+0.2672 [+0.2015, +0.3348]
	TE-G-SAGE + features	+0.0990 [+0.0425, +0.1601]	+0.1502 [+0.0749, +0.2268]

Table 10: Decomposition of the NF-UNSW-NB15 gap against TE-G-SAGE. Point estimates only. Shares are of the total gap to the feedback model, except the final row, whose share is of the smaller gap to the fusion model and names its own denominator. Feedback has no separated advantage over the trained head alone (Table 4).

Step	macro-F1	attributable to	share
TE-G-SAGE as published	0.3841	—	—
+ fair tuning (refit)	0.5842	+0.2001, graph scale and schedule	44.5%
+ our ten centralities	0.7196	+0.1354, feature engineering	30.1%
our feedback model rung	0.8341	+0.1145, architecture	25.4%
(our fusion model rung)	0.7793	+0.0597 over the same base	15.1% of 0.3952



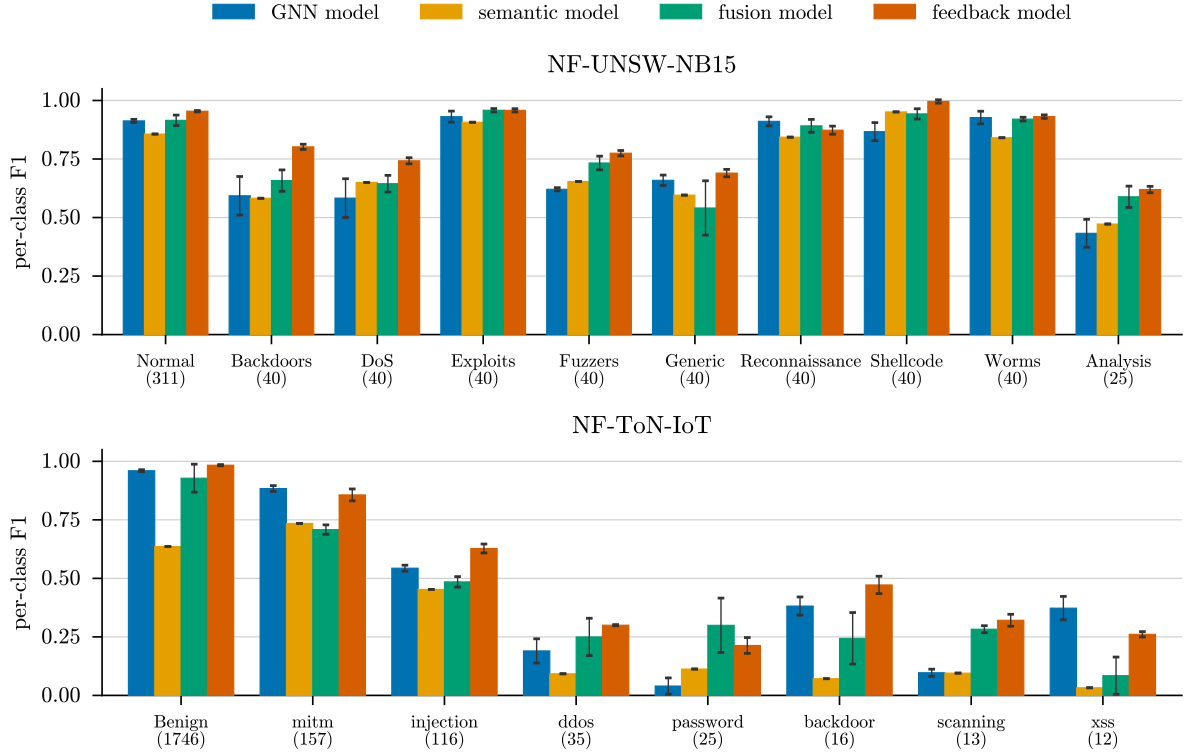
Point estimates only; every level is a three-seed mean and no step carries an interval or a separation claim.

Figure 9: The NF-UNSW-NB15 margin against TE-G-SAGE as published, decomposed. Roughly 45% of the gap is our choice of how to tune, 30% is our ten node centralities, and 25% is the architecture. The tick on the final increment marks where the fusion model lands on the same scale. Every level is a three-seed mean; no step carries an interval and no step is a separation claim.

4.7 Per-class behaviour

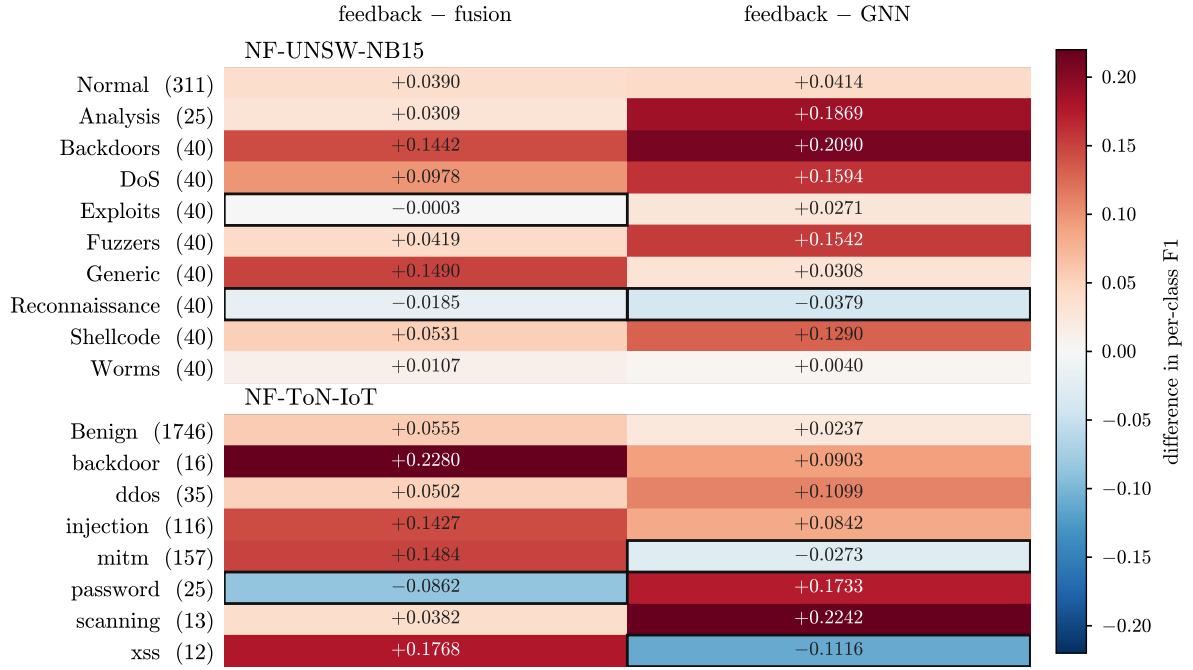
Figures 10 and 11, with Table 15, show where the aggregate scores come from. Feedback exceeds fusion on eight of ten NF-UNSW-NB15 classes and seven of eight NF-ToN-IoT classes. Against the GNN, it is higher on nine of ten and six of eight, respectively. Its gains include +0.2090 for NF-UNSW-NB15 Backdoors and +0.1594 for DoS, and +0.0903 for NF-ToN-IoT `backdoor` and +0.2242 for `scanning`. Losses against the GNN include NF-ToN-IoT `mitm` and `xss`. Feedback also trails fusion on `password`, at 0.2134 against 0.2996. The trained head alone remains slightly higher in overall mean macro-F1, without a separated difference from feedback.

The gains and losses occur in different classes, so a higher macro-F1 does not mean every attack is detected better. Several classes contain only 12–40 edges, and changing a small number of predictions can move their F1 substantially. None of the per-class differences has a confidence interval. The fusion stage illustrates the trade-off: on NF-ToN-IoT it raises `password` from the GNN’s 0.0401 to 0.2996, while `mitm` falls from 0.8838 to 0.7082 and `xss` from 0.3728 to 0.0844. These observations identify where models differ; they do not establish why fusion loses overall or show that imbalance alone causes those losses. Appendix B.1 retains the class counts and detailed scores.



Three-seed means with training-seed standard deviations; classes are ordered by edge count, given under each label. The feedback model consults the trained head. No per-class difference carries an interval, and on the smallest classes a single edge moves F1 substantially.

Figure 10: Per-class F1 for the four rungs, both datasets, with classes ordered by edge count and the count under each label. The feedback model is the canonical one, consulting the trained head. Error bars are one standard deviation over three training seeds at a fixed fold partition. No per-class value carries an interval, and on the smallest classes a single edge moves F1 substantially.



Three-seed means. Class sizes in brackets. A black outline marks a loss. No per-class difference carries an interval, and on classes of 12 to 40 edges a single edge moves F1 substantially.

Figure 11: Per-class differences between the feedback model and the two rungs below it, three-seed means, with class sizes in brackets. The scale is centred at zero and a black outline marks a loss. No per-class difference carries an interval, and on classes of 12 to 40 edges a single edge moves F1 substantially.

4.8 Knob selection

Validation-only selection chose consultation percentile $k = 29$ and injection scale 2.0 on NF-UNSW-NB15, and $k = 16$ and scale 20.0 on NF-ToN-IoT. The confidence gate retains half of the flagged edges, giving effective consultation rates of 14.5% and 8.0%. At these settings, 190 of 656 edges are flagged and 95 gated on NF-UNSW-NB15; 341 of 2,127 are flagged and 170 gated on NF-ToN-IoT. Across the 21 percentile candidates, mean validation macro-F1 ranges from 0.8238 to 0.8277 and from 0.5170 to 0.5232. The six scale candidates span 0.8252–0.8277 and 0.5196–0.5232. Appendix B.2 reports the sweep summaries.

Both curves are flat relative to the observed training-seed variation, so the selected maximum should not be described as a well-established optimum. NF-ToN-IoT’s scale is at the upper boundary of the tested range, and these runs cannot determine whether a better value lies beyond it. We therefore call the settings *selected*, not *tuned*. Settings were selected separately for each dataset and consultant. The prototype-feedback comparison used percentiles 31 and 25 at the same injection scales, which is why the change between feedback configurations cannot be attributed solely to replacing the consultant.

5 Discussion

This section interprets the GLASS evaluation. It first answers the research questions and compares the patterns observed on the two datasets. It then examines what the consultant and feedback mechanism contribute, states the limitations, and finally identifies the experiments needed to resolve the remaining questions.

5.1 Answers to the research questions

RQ1. The GNN and prototype semantic model are close on NF-UNSW-NB15, at 0.7437 and 0.7353, but farther apart on NF-ToN-IoT, at 0.4336 and 0.2785. This does not establish that the semantic embeddings contain little useful information. A trained head on the same frozen embeddings reaches 0.8374 and 0.5081. The choice of classifier therefore matters when judging the semantic branch. Table 4 reports the head separately from the four-stage ladder.

RQ2. The two comparisons give different answers. Fusion does not establish a gain over the GNN on either dataset: its two-level differences are $+0.0370 [-0.0212, +0.0929]$ on NF-UNSW-NB15 and $-0.0235 [-0.0910, +0.0602]$ on NF-ToN-IoT. The NF-UNSW-NB15 difference is separated under the secondary seed-matched procedure, but not under the primary procedure that includes independent training-seed variation. Against the prototype semantic model, fusion does establish a gain on both datasets: $+0.0443 [+0.0067, +0.0795]$ and $+0.1299 [+0.0704, +0.2054]$, respectively. Both secondary intervals also exclude zero (Table 5). These additional intervals were calculated from the saved predictions using the same bootstrap procedures, without retraining or changing model selection. Thus GLASS fusion improves over the prototype semantic stage, but RQ2’s stronger claim of an improvement over both single-branch stages is unsupported. This comparison concerns the prototype scorer used in the ladder, not the stronger trained semantic head.

RQ3. The complete trained-head feedback model improves over the graph-only control and the fusion stage, but not over the trained semantic head alone by a separated margin. With output fusion disabled, the prototype’s differences against control are -0.0046 and $+0.0099$, neither separated. The trained head gives $+0.0007$ on NF-UNSW-NB15, also not separated, and a separated loss of -0.0378 on NF-ToN-IoT. This isolation experiment used earlier consultation percentiles, so it does not directly quantify injection at the final settings. Together, the experiments show the value of

semantic predictions in the complete model but leave an additional injection benefit unestablished.

RQ4. Model behaviour changes across the two graphs, but the cause cannot be assigned to imbalance alone. The graphs also differ in size, density and repeated endpoint pairs. Topology constrains attention injection: parallel edges share endpoint representations, while an incoming neighbourhood of one makes its attention weight fixed. Consultant choice also matters, as the prototype and trained head behave differently on the uncertain edges. The per-class results show where fusion gains and loses; they do not isolate why those changes occur.

RQ5. Trained-head feedback is separated above the faithful re-trained baselines on both datasets, while its difference from the head alone is not separated. These are comparisons on our aggregated representation, not against published benchmark scores. On NF-ToN-IoT, the GNN and fusion stages are not separated from E-GraphSAGE with its published settings. On NF-UNSW-NB15, the point-estimate decomposition associates roughly 45% of the gap from TE-G-SAGE to feedback with validation selection, 30% with node features and 25% with the remaining model change. Those increments have no confidence intervals and should not be read as a general architectural ranking.

5.2 Patterns across the two imbalance regimes

The datasets were selected to represent different class distributions. Figure 2 shows the graph-level imbalance and distinguishes it from imbalance among the evaluated classes. Fusion has a higher mean macro-F1 than the GNN on NF-UNSW-NB15 and a lower mean on NF-ToN-IoT. Neither difference is separated under the primary bootstrap. The comparison establishes this change in ordering, but two datasets cannot isolate imbalance from their other differences.

The NF-ToN-IoT class results show a trade-off. Fusion raises `password` from the GNN’s 0.0401 to 0.2996, while `mitm` falls from 0.8838 to 0.7082 and `xss` from 0.3728 to 0.0844. Explaining the net loss would require further controlled tests. We therefore do not attribute it to a particular gate behaviour or to imbalance alone. Small class counts also make these point estimates sensitive to a few predictions.

Macro-F1 and class-weighted focal loss address the imbalance in different ways: the metric gives each evaluated class equal importance, while the loss changes the training emphasis. Additional oversampling was measured and was harmful when combined with the existing class weights, so it is disabled. These choices do not eliminate the uncertainty associated with rare classes.

The prototype’s low NF-ToN-IoT score is also insufficient evidence of an encoder limitation. The trained head performs better on the same frozen embeddings, showing that the readout affects the result. However, no experiment adapted CySecBERT to the templated flow descriptions. The effects of the readout, data imbalance and encoder adaptation have therefore not been fully separated.

5.3 Where the feedback model’s gain comes from

The feedback model has two semantic routes. Injection changes the edge representation before another graph pass. Output fusion gives the final classifier direct access to the semantic predictions. A gain from the complete model cannot, by itself, tell us which route helps. The trained head alone has a slightly higher mean macro-F1 than feedback on both datasets, with no separated difference between them.

The isolation diagnostic in Table 7 disables output fusion. An oracle with access to the true label improves through edge injection on both graphs, demonstrating that this route can change accuracy under the tested conditions. Neither realistic consultant produces a separated improvement there. The trained head even produces a separated loss on NF-ToN-IoT. This is evidence that a stronger standalone classifier need not supply better advice for injection.

The prototype treatments offer further diagnostic evidence. Changing selection, temperature, reliability weighting, advice format and cross-fitting did not establish the intended improvement. In the format experiment, larger input logits led to smaller injected magnitudes. Cross-fitting changed injected magnitudes more than the learned trust scalar. Appendix A records these measurements and their single-seed or multi-seed scope. They motivate investigating how advice is used, without proving that every alternative mechanism would fail.

The most direct conclusion is that the complete system benefits from strong semantic predictions, while the experiments do not establish a further gain from injection. Because the isolation diagnostic used earlier consultation percentiles, a matched diagnostic at the final settings would be needed to attribute the final system’s performance more precisely. The head-alone comparison remains the direct check on whether the full system adds measurable value beyond its consultant.

5.4 What the configuration buys, and what it does not

The trained-head configuration gives separated improvements over the GNN, semantic and fusion stages and the faithful re-trained baselines on both datasets. It does not establish an improvement over the trained head alone. This distinction explains why the

report includes both a four-stage architectural comparison and a separate component comparison. Omitting the consultant’s standalone result would make the feedback stage appear to add more than the measurements support.

The choice of trained head is supported by its stronger standalone predictions and its higher accuracy on the consulted edges. Earlier prototype treatments failed to establish the intended gain, which motivated this change. However, the consultant and consultation percentile changed together, and the difference between the two feedback configurations is separated only on NF-UNSW-NB15 under the primary procedure. We can justify the choice without claiming that the experiments isolate its causal effect or establish a feedback gain over its own component.

The contribution is the implementation and evaluation of an edge-level structural-semantic system, including tests that distinguish a useful consultant from a useful injection mechanism. The underlying ideas of graph-language feedback, uncertainty-based consultation and learned routing come from the prior work reviewed in Section 2. The measurements here establish both positive comparisons and limits on what can be credited to the feedback mechanism.

5.5 Limitations

The following limitations determine how broadly the results can be interpreted.

1. **Controlled graph construction.** The deployment restriction and the changes needed to address it are developed in Section 3.2. The present results must be interpreted within that scope.
2. **A fixed fold partition and small classes.** Training seeds vary, but the fold partition does not. The reported uncertainty therefore excludes partition variation. Several evaluated classes contain only 12–40 edges, and no per-class difference has a confidence interval.
3. **Training advice differs from test advice.** Each fold’s semantic head has seen its training-edge labels, so its advice on those edges is in-fold. The reported cross-fitting diagnostics do not establish a reliable remedy. Their settings and evidence are recorded in Appendix A.
4. **Flat selection curves and a boundary value.** The feedback settings are selected from curves that are flat relative to training variation. NF-ToN-IoT’s scale is at the upper search boundary. Performance outside the tested range is unknown.

5. **Fixed consultant advice across feedback seeds.** Feedback training reuses the seed-42 head predictions. Its reported standard deviation excludes the variation that would arise from retraining its consultant. A fully re-seeded pipeline was not measured.
6. **Scope of the isolation experiment.** Its consultation percentiles differ from the final configuration. Its paired intervals also use fifteen (seed, fold) pairs, which are not independent because their training sets overlap. These diagnostics should not be treated as a matched decomposition of the final model.
7. **Baseline representation and scoring differences.** E-GraphSAGE is not separated from our GNN or fusion stage on NF-ToN-IoT. The reported fusion and baseline runs also allowed predictions of excluded classes. Appendix B.5 bounds the possible cost; the code was corrected without replacing the reported measurements.
8. **An unadapted semantic encoder.** CySecBERT remains frozen, and no fine-tuned or domain-adapted version was evaluated on the flow templates. The current results cannot determine what such adaptation would add.
9. **Unisolated structural contributions.** No run disables message passing, no flat classifier is evaluated on the same edge and centrality features, and no ablation compares the three weighted branches with one unweighted branch. The value of these structural design choices is therefore not established separately.

5.6 What follows from this

The first experimental priority is a matched comparison of the trained head alone, output fusion without injection, and the complete feedback model at the final consultation settings. It should vary the consultant’s training seed as well as the feedback model’s seed. This would directly test whether the added mechanism contributes beyond access to strong semantic predictions.

A second direction is the reliability of advice on the edges selected for consultation. The oracle diagnostic shows that correct advice can help through the channel, but the trained-head diagnostic shows that stronger overall classification need not improve injection. A per-edge reliability estimate or a supervised trust function could be tested against the existing scalar, with all comparisons using the same settings and controls.

Deployment-valid evaluation requires the graph and temporal protocol changes described in Section 3.2. It is a separate experiment on a changed task. Advice conditioned on the graph model’s current predictions would also change the question: the present

consultant reads only an edge’s text, and its advice does not change when the graph model changes its belief.

5.7 Scope, and the experiments this work did not reach

The remaining experiments include repeating the evaluation with new fold partitions, extending the boundary-selected scale range, and adapting the frozen encoder to the flow descriptions. None was run for the results reported here. Their absence is a limit on the evidence, rather than a reason to assume that they would improve the scores.

The structural branch needs its own controls. Disabling message passing and fitting a flat classifier to the same aggregated edge attributes and endpoint centralities would test whether graph propagation adds useful information. Comparing the three weighted encoders with a single unweighted encoder would test the branch split. These comparisons address the supervisor’s open methodological questions and remain future work.

6 Conclusion

This section first revisits the motivation for GLASS, then summarises its design and results, and finally identifies the limitations and next steps.

IoT networks produce traffic whose security depends on both individual events and patterns across communicating devices. Machine learning can classify connections from numerical measurements, and deep learning can learn representations of those measurements. GNNs add a way to model communication context, while pretrained language models offer representations of textual event descriptions. Combining these approaches is a reasonable design direction, but it also raises a practical question: does their interaction improve detection beyond the strongest individual component?

This study addressed that question through GLASS (Graph-Language Adaptive Semantic-Structural fusion). GLASS converts aggregated traffic into a directed graph and a corresponding set of templated edge descriptions. A graph attention network learns structural representations, and frozen CySecBERT supplies semantic embeddings. An adaptive gate combines the two views across edges and feature dimensions. The feedback stage adds uncertainty-guided consultation, injecting semantic advice before another graph update and using semantic predictions again at the output. The consultant reads the edge’s text; it does not revise its advice in response to the graph model’s changing belief.

GLASS was evaluated on NF-UNSW-NB15 and NF-ToN-IoT using pooled five-fold out-of-fold predictions at three training seeds and a fixed fold partition. The comparison included the GNN, prototype semantic, fusion and feedback stages, both prototype and trained-head consultants, the trained head alone, and published detectors re-trained on the same aggregated representation. This design makes it possible to distinguish the performance of the complete configuration from the evidence supporting its individual mechanisms.

The trained-head feedback configuration is separated above the GNN, prototype semantic and fusion stages and the faithful re-trained baselines on both datasets. However, the trained head alone has slightly higher mean macro-F1 on both, and its difference from feedback is not separated. Its stronger standalone predictions support the choice of trained head over the prototype readout, but the complete GLASS feedback design has not established an additional benefit over that consultant. Changing the consultant also changed the consultation percentile, so the difference between feedback configurations cannot be assigned to either change alone.

The mechanism diagnostics reinforce this distinction. With output fusion disabled, neither realistic consultant provides a separated injection gain at the earlier consultation settings tested; the trained head produces a separated loss on NF-ToN-IoT. An

oracle supplied with true labels improves through the same channel, showing that useful information can pass through it. Attention injection changes no predicted classes between passes in the tested configurations. Fusion is separated above the prototype semantic model on both datasets, but not above the GNN. It therefore does not establish an improvement over both single-branch stages. These findings show why a more elaborate interaction cannot be assumed to produce a better detector.

The conclusions about GLASS remain within the controlled graph-construction scope defined in Section 3.2: labels help determine the aggregated edges, so the current evaluation does not represent deployment on unlabelled traffic. Training-seed variation is measured at one fold partition, small classes limit interpretation, and the mechanism diagnostic does not use the final consultation settings. The next steps are matched mechanism tests at those settings, broader uncertainty measurement, controls for message passing and the weighted branch split, and evaluation with label-independent traffic aggregation. GLASS establishes a reproducible framework for testing these questions and demonstrates the value of a trained semantic classifier, while leaving the additional value of feedback open.

Acknowledgements

I thank my supervisors, Dr. Hussein Hazimeh and Dr. Fouad Al Tfaily, for their guidance throughout this work. Dr. Fouad Al Tfaily supervised the day-to-day work at CESI LINEACT. I thank CESI LINEACT and the Lebanese University for hosting it. The two datasets are used in the NetFlow-standardised forms released by their authors, and the two baseline architectures were re-implemented from their published descriptions and released code.

Several of this report’s findings exist because a claim was checked and did not survive. That habit cost time and improved the work, and the record of what was withdrawn is kept alongside the record of what stood.

This work was carried out between March and September 2026. The period was one of war. Displacement and unreliable connectivity interrupted the work repeatedly. The scope reported here is what that period allowed. Section 5.7 sets out the experiments it did not reach and the order in which they would be run.

Statement on the use of AI assistance

AI assistance was used for drafting and editing prose in this report, for writing and refactoring pipeline and analysis code, and for building the scripts that audit the manuscript’s numbers against the committed result artifacts. Every experiment was designed and run by the author. Every number reported here comes from a committed artifact produced by those runs, and no figure, table or quantitative claim was generated from a model’s recollection. The interpretation of the results, the decisions about what to measure, what to retract and what to report as negative, and the final text are the author’s own.

Data and code availability

This study uses the NetFlow-standardised NF-UNSW-NB15 and NF-ToN-IoT datasets described by Sarhan et al. [28], derived from UNSW-NB15 [23] and TON_IoT [22]. The cited dataset publications identify their sources. The project repository is <https://github.com/alialzein01/LLM-G-IDS>. Appendix C lists the environment, reproduction commands and result artifacts; local trace files required to recalculate the churn diagnostic are identified in Appendix A.5.

References

- [1] John Arévalo, Tamar Solorio, Manuel Montes y Gómez, and Fabio A. González. Gated multimodal units for information fusion, 2017. arXiv:1702.01992.
- [2] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don'ts of machine learning in computer security. In *31st USENIX Security Symposium*. USENIX Association, 2022. arXiv:2010.09470.
- [3] Markus Bayer, Philipp Kuehn, Ramin Shanehsaz, and Christian Reuter. Cy-SecBERT: A domain-adapted language model for the cybersecurity domain. *ACM Transactions on Privacy and Security*, 27(2):1–20, 2024. doi: 10.1145/3652594.
- [4] Azadeh Beiranvand and S. Mehdi Vahidipour. Integrating structural and semantic signals in text-attributed graphs with BiGTex. *Machine Learning with Applications*, 24:100921, 2026. doi: 10.1016/j.mlwa.2026.100921.
- [5] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2105.14491.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pages 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1423.
- [7] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994. doi: 10.1201/9780429246593.
- [8] Yasir Ali Farrukh, Syed Wali, Irfan Khan, and Nathaniel D. Bastian. XG-NID: Dual-modality network intrusion detection using a heterogeneous graph neural network and large language model. *Expert Systems with Applications*, 287:128089, 2025. doi: 10.1016/j.eswa.2025.128089.
- [9] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. arXiv:1903.02428.
- [10] Lingbing Guo, Zhuo Chen, Jiaoyan Chen, Qiang Zhang, and Huajun Chen. DET: A dual-encoding transformer for relational graph embedding. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 4685–4696, 2024. doi: 10.63317/29cd7cmbyvh5.

- [11] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017. arXiv:1706.02216.
- [12] Hong Huang, Xingxing Zhang, Ye Lu, Ze Li, and Shaohua Zhou. BSTFNet: An encrypted malicious traffic classification method integrating global semantic and spatiotemporal features. *Computers, Materials & Continua*, 78(3):3929–3951, 2024. doi: 10.32604/cmc.2024.047918.
- [13] Jaewan Lee, Changyoung Park, Hongjun Yang, Sungbin Lim, Woohyung Lim, and Sehui Han. CAST: Cross attention based multimodal fusion of structure and text for materials property prediction, 2025. arXiv:2502.06836.
- [14] Haolin Li, Shuyang Jiang, Lifeng Zhang, Siyuan Du, Guangnan Ye, and Hongfeng Chai. RAGFormer: Learning semantic attributes and topological structure for fraud detection, 2024. arXiv:2402.17472.
- [15] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 2999–3007, 2017. doi: 10.1109/ICCV.2017.324.
- [16] Yuxiao Lin, Yuxian Meng, Xiaofei Sun, Qinghong Han, Kun Kuang, Jiwei Li, and Fei Wu. BertGCN: Transductive text classification by combining GNN and BERT. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 1456–1462, 2021. doi: 10.18653/v1/2021.findings-acl.126.
- [17] Wai Weng Lo, Siamak Layeghy, Mohanad Sarhan, Marcus Gallagher, and Marius Portmann. E-GraphSAGE: A graph neural network based intrusion detection system for IoT. In *IEEE/IFIP Network Operations and Management Symposium (NOMS)*, pages 1–9, 2022. doi: 10.1109/NOMS54207.2022.9789878. Code: github.com/waimorris/E-GraphSAGE.
- [18] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.
- [19] Donald Loveland, Yao-An Yang, and Danai Koutra. Glance for context: Learning when to leverage LLMs for node-aware GNN-LLM fusion, 2025. arXiv:2510.10849.
- [20] Riko Luša, Damir Pintar, and Mihaela Vranić. TE-G-SAGE: Explainable edge-aware graph neural networks for network intrusion detection. *Modelling*, 6(4):165, 2025. doi: 10.3390/modelling6040165. Code: github.com/Ricco555/TE-G-SAGE-XAI.
- [21] Hejamadi Rama Moorthy, N. J. Avinash, N. S. Krishnaraj Rao, K. R. Raghunandan, Radhakrishna Dodmane, Jeremy Joseph Blum, and Lubna A. Gabralla.

- Dual stream graph augmented transformer model integrating BERT and GNNs for context aware fake news detection. *Scientific Reports*, 15:25436, 2025. doi: 10.1038/s41598-025-05586-w.
- [22] Nour Moustafa. A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. *Sustainable Cities and Society*, 72:102994, 2021. doi: 10.1016/j.scs.2021.102994.
 - [23] Nour Moustafa and Jill Slay. UNSW-NB15: A comprehensive data set for network intrusion detection systems. In *2015 Military Communications and Information Systems Conference (MilCIS)*, pages 1–6, 2015. doi: 10.1109/MilCIS.2015.7348942.
 - [24] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
 - [25] Feargus Pendlebury, Fabio Pierazzi, Roberto Jordaney, Johannes Kinder, and Lorenzo Cavallaro. TESSERACT: Eliminating experimental bias in malware classification across space and time. In *28th USENIX Security Symposium*. USENIX Association, 2019. arXiv:1807.07838.
 - [26] Yiran Qiao, Xiang Ao, Yang Liu, Jiarong Xu, Xiaoqian Sun, and Qing He. LOGIN: A large language model consulted graph neural network training framework. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining (WSDM)*, pages 232–241. ACM, 2025. doi: 10.1145/3701551.3703488.
 - [27] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410.
 - [28] Mohanad Sarhan, Siamak Layeghy, Nour Moustafa, and Marius Portmann. Net-Flow datasets for machine learning-based network intrusion detection systems. In *Big Data Technologies and Applications*, pages 117–135. Springer International Publishing, 2021. doi: 10.1007/978-3-030-72802-1_9.
 - [29] Karen Scarfone and Peter Mell. Guide to intrusion detection and prevention systems (IDPS). Technical Report NIST Special Publication 800-94, National Institute of Standards and Technology, 2007.

- [30] Aarush Sinha. GMLM: Bridging graph neural networks and language models for heterophilic node classification, 2025. arXiv:2503.05763.
- [31] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy*, pages 305–316, 2010. doi: 10.1109/SP.2010.25.
- [32] Safal Thapaliya, Zehong Wang, Jiazheng Li, Ziming Li, Yanfang Ye, and Chuxu Zhang. Semantic refinement with LLMs for graph representations, 2025. arXiv:2512.21106v1. The v1 text names the framework DAS; the April-2026 revision renames it GES.
- [33] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *6th International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.10903.
- [34] Zhaoyan Wang, Zheng Gao, Arogya Kharel, and In-Young Ko. Are LLMs better GNN helpers? rethinking robust graph learning under deficiencies with iterative refinement, 2025. arXiv:2510.01910; system named RoGRAD.
- [35] Lan Xiong, Liang Wan, and Jingxia Ren. Two-branch intrusion detection method based on fusion of deep semantic and statistical features. *Computers, Materials & Continua*, 87(3):18, 2026. doi: 10.32604/cmc.2026.076986.
- [36] Haotong Yang, Xiyuan Wang, Qian Tao, Shuxian Hu, Zhouchen Lin, and Muhan Zhang. GL-Fusion: Rethinking the combination of graph neural network and large language model, 2024. arXiv:2412.06849.
- [37] Zhonghao Zhan, Huichi Zhou, and Hamed Haddadi. Poster: Enhancing GNN robustness for network intrusion detection via agent-based analysis, 2025. arXiv:2506.20806; poster at IEEE EuroS&P 2025.
- [38] Jingkang Zhang, Xinjian Fan, and Zhixin Zhao. A hybrid intrusion detection model based on dynamic spatial-temporal graph neural network in in-vehicle networks. *Scientific Reports*, 15(1):34736, 2025. doi: 10.1038/s41598-025-18401-3. Model named GCN-2-Former.

A Negative mechanism results

This appendix first records the prototype treatment conditions and injection magnitudes, then gives class reliability and treatment details, and finally documents the isolation diagnostics.

This appendix holds the measurements behind Section 4.4. They are reported in full because the case for changing the feedback model’s consultant rests on them, and because each one closes a line of investigation that would otherwise look open.

A.1 Conditions A, B and C

Three configurations of the feedback model were run at three seeds on both datasets, with the fold partition fixed. Condition A is the canonical prototype-consultant feedback model. Condition B trains the selector head whose entropy ranks edges and calibrates the consultant’s temperature per fold instead of pinning it. Condition C re-selects the two knobs on top of B. The GNN model and fusion model rungs are unaffected by any of the three and are shown for reference in Table 11.

Table 11: Pooled out-of-fold macro-F1, mean over three seeds, by condition.

Dataset	Rung	A (canonical)	B (signals fixed)	C (B, re-selected)
UNSW	feedback model	0.7644	0.7610	0.7809
	fusion model	0.7793	0.7793	0.7793
	GNN model	0.7437	0.7437	0.7437
ToN	feedback model	0.4521	0.4147	0.4630
	fusion model	0.4102	0.4102	0.4102
	GNN model	0.4336	0.4336	0.4336

The decision rule was fixed before the runs. It returns the same verdict in all three conditions: the feedback model is not separated from the fusion model on either dataset. On NF-UNSW-NB15 the two-level interval for the feedback model minus the fusion model is -0.0145 $[-0.0556, +0.0276]$ in A, -0.0181 $[-0.0596, +0.0242]$ in B and $+0.0018$ $[-0.0406, +0.0444]$ in C. On NF-ToN-IoT it is $+0.0433$ $[-0.0376, +0.1199]$, $+0.0065$ $[-0.0782, +0.0788]$ and $+0.0531$ $[-0.0360, +0.1340]$. Condition C carries the highest point estimate on both datasets. That is not a separation, and its selection curves are flat, with NF-UNSW-NB15’s argmax landing on the boundary of the swept range at $k = 35$.

Both defects were genuinely removed in B. That is what makes the negative result informative rather than a failed implementation. On NF-UNSW-NB15 the selector head’s macro-F1 moved from the range 0.019 to 0.036 up to the range 0.646 to 0.684, mean

disagreement fell from 0.8975 to between 0.524 and 0.548, the consultant’s maximum softmax probability rose from 0.102 to 0.487, and the temperature fell from about 9.4 to 0.105. On NF-ToN-IoT the selector head moved from 0.029 to 0.065 up to 0.293 to 0.333, disagreement fell from 0.8988 to between 0.580 and 0.638, maximum probability rose from 0.101 to 0.500, and temperature fell from about 9.5 to 0.030. A better-calibrated consultant and a genuinely uncertainty-ranked selector made the feedback model worse on both datasets.

A.2 What the canonical feedback model was actually injecting

Table 12 gives the mean absolute magnitude added to the 39-wide encoded edge vector on flagged edges, per seed. The encoded features have unit variance, so these numbers are directly interpretable as a fraction of a standard deviation.

Table 12: Injected bias magnitude on flagged edges, per seed, by condition.

Condition	NF-UNSW-NB15	NF-ToN-IoT
A (canonical)	0.077 / 0.060 / 0.061	0.023 / 0.021 / 0.023
B (signals fixed)	1.228 / 1.171 / 1.218	2.796 / 2.854 / 2.762
C (B, re-selected)	1.103 / 1.100 / 1.158	2.845 / 2.893 / 2.992

Under the canonical prototype configuration the mechanism was injecting between 0.02 and 0.08 of a standard deviation. Calibrating the temperature raised that by a factor of roughly 17 on NF-UNSW-NB15 and roughly 125 on NF-ToN-IoT. Two things follow. The prototype feedback model’s advantage over feedback-off was late fusion rather than feedback. The injection scale is also close to redundant with the learned bias strength, since condition C moved the scale from 2.0 to 5.0 and from 20.0 to 10.0 while the end-to-end magnitude barely moved.

A.3 Per-class reliability of the embedding-only consultant

The reliability-weighting treatment computes, for each class, the precision of the prototype consultant’s argmax on the fold’s training edges. The weights are a more useful result than the macro-F1 deltas they produced, because they quantify the consultant’s weakness per class from training labels alone. Table 13 gives them.

On NF-UNSW-NB15 six of ten classes sit at or above 0.9 and the minimum is 0.662, so reweighting has little to reorder. On NF-ToN-IoT six of ten sit below 0.226: `ransomware` at 0.020, `dos` at 0.021, `backdoor` at 0.071, `xss` at 0.089, `scanning` at 0.095 and `ddos` at 0.179, with `password` a seventh sitting at 0.226 itself. That is the same

Table 13: Mean per-class precision of the prototype consultant on training edges, over five folds. Each dataset’s classes are listed in its own label-mapping order.

Index	NF-UNSW-NB15		NF-ToN-IoT	
	class	precision	class	precision
0	Normal	1.000	Benign	1.000
1	Analysis	0.852	backdoor	0.071
2	Backdoors	0.976	ddos	0.179
3	DoS	0.893	dos	0.021
4	Exploits	0.961	injection	0.570
5	Fuzzers	0.662	mitm	0.740
6	Generic	0.895	password	0.226
7	Reconnaissance	0.984	ransomware	0.020
8	Shellcode	1.000	scanning	0.095
9	Worms	0.960	xss	0.089

weakness the isolation experiment measures as the binding constraint, now quantified per class.

A.4 The five treatments

Table 14: Five treatments applied to the embedding-only consultant. None was kept. Rows marked seed 42 carry no interval across seeds and are single-seed orderings only.

Treatment	What it changed	Result	Kept
Selector-head supervision	the ranking head is trained as a classifier	3 seeds: feedback model worse on both datasets	no
Consultant temperature	per-fold calibration instead of a pinned T	3 seeds: feedback model worse, jointly with the above	no
Reliability weighting	gate and magnitude scaled by per-class train precision	seed 42: +0.0057 / +0.0094, below the 0.02 rule	no
Advice format	one-hot and softmax at the oracle’s ± 4 nats	seed 42: every arm below baseline on both	no
Cross-fitted advice	train rows replaced by inner out-of-fold logits	seed 42: −0.0246 / −0.0338 against in-fold	no

The advice-format arm produced a counter-intuitive measurement worth stating on its own. Handing the consultant the oracle’s saturated format made the input magnitude larger and the injected magnitude smaller, from 0.0767 to 0.0183 on NF-UNSW-NB15 and from 0.0234 to 0.0202 on NF-ToN-IoT. The projection is learned, so it adapts to a larger and lower-variance input by injecting less. Whatever the oracle’s advantage is, it is not the shape of the vector it presents.

The cross-fitting arm tested the feedback model’s trust directly. If in-fold optimism in the advice were what the feedback model was exploiting, honest advice should lower the scalar the feedback model learns for how hard to push. Across four settings the learned trust scalar moved by at most 0.0205, from 1.5722 to 1.5664 and 1.5227 to 1.5022 on NF-UNSW-NB15 and from 1.4768 to 1.4652 and 1.4842 to 1.4899 on NF-ToN-IoT, moving the wrong way in the last of those. What did move was the injected magnitude, from 1.3430 to 1.1752 and from 1.3291 to 1.0745. The model absorbs the change in the projection rather than in the parameter that is supposed to represent trust.

A.5 Isolation diagnostics and prediction changes

The isolation measurements use the earlier consultation percentiles 31 on NF-UNSW-NB15 and 25 on NF-ToN-IoT, with output fusion disabled. They should not be read as a direct ablation of the final percentiles 29 and 16. The paired intervals resample (seed, fold) pairs and differ from the two-level edge bootstrap used for the model ladder.

The oracle headroom, computed from the plain three-seed means, is +0.0345 on NF-UNSW-NB15 and +0.1090 on NF-ToN-IoT. The captured shares in Table 7 use these values, rather than the paired-bootstrap mean differences. The prototype captures

−4.5% and 11.9%, neither separated from zero; the trained head captures 12.5% and −38.0%, with the latter a separated loss.

The NF-UNSW-NB15 headroom comes with a caveat. At seed 42 alone the oracle arm scores 0.7522 against a control of 0.7543, a headroom of −0.0021, so a captured share is not computable there. Because the +0.0345 is a mean over three seeds that do not agree in sign, single-seed shares on NF-UNSW-NB15 are not reported.

A cross-fitted trained-head diagnostic at seed 42 moves the difference against control from −0.0230 to −0.0654 on NF-UNSW-NB15 and from −0.0165 to +0.0070 on NF-ToN-IoT. The single-seed result does not establish a reliable improvement.

Churn, the fraction of edges whose predicted class changes between consecutive passes, measured in the canonical feedback model over every edge in the graph, training edges included, is 0.1372 on NF-UNSW-NB15 and 0.1246 on NF-ToN-IoT as three-seed means, with per-seed values from 0.0938 to 0.1675 and from 0.1070 to 0.1402. The injection path is alive and moves predictions on every pass. This measures prediction changes, not accuracy gains. The churn figures are in `results/feedback_churn.json`, and Section 4.5 contrasts them with the attention variant, whose churn is exactly zero.

The churn summary is saved in `results/feedback_churn.json`, but its source traces are in the local, untracked `tmp/multiseed_v2` directory. The summary is available; independent recalculation of these particular values requires those traces.

B Supporting tables

This appendix first provides the per-class and selection tables, then examines consultant complementarity, and finally gives detailed model comparisons and baseline qualifications.

B.1 Per-class F1

Table 15: Per-class F1, mean over three seeds, with edge counts. The feedback model is the canonical one, consulting the trained head. NF-ToN-IoT’s **dos** and **ransomware** are excluded from the metric and omitted here, and F1 is computed on the rows the metric scores, so each column’s mean is that rung’s macro-F1 in Table 3.

Class	edges	GNN	fusion	feedback
<i>NF-UNSW-NB15</i>				
Normal	311	0.9129	0.9152	0.9543
Analysis	25	0.4327	0.5886	0.6196
Backdoors	40	0.5929	0.6578	0.8019
DoS	40	0.5831	0.6446	0.7424
Exploits	40	0.9309	0.9583	0.9580
Fuzzers	40	0.6207	0.7330	0.7749
Generic	40	0.6591	0.5409	0.6899
Reconnaissance	40	0.9110	0.8916	0.8731
Shellcode	40	0.8669	0.9428	0.9959
Worms	40	0.9272	0.9205	0.9311
<i>NF-ToN-IoT</i>				
Benign	1746	0.9599	0.9280	0.9835
backdoor	16	0.3817	0.2439	0.4720
ddos	35	0.1901	0.2498	0.3000
injection	116	0.5436	0.4851	0.6277
mitm	157	0.8838	0.7082	0.8566
password	25	0.0401	0.2996	0.2134
scanning	13	0.0969	0.2829	0.3211
xss	12	0.3728	0.0844	0.2612

B.2 Knob selection curves

Table 16 summarises the validation-only searches. The ranges show how little the score changes across the tested settings and identify the boundary-selected scale.

Table 16: Knob selection curves for the trained-head consultant, mean validation macro-F1 over selection seeds 42, 1 and 2. Every span is smaller than the rungs’ own seed-to-seed spread, so the argmax is not distinguishable from its neighbours.

Dataset	Knob	candidates	range	validation macro-F1	span	selected	on boundary
NF-UNSW-NB15	k (entropy percentile)	21	15–35	0.8238–0.8277	0.0038	29	no
NF-UNSW-NB15	injection scale	6	0.5–20	0.8252–0.8277	0.0025	2	no
NF-ToN-IoT	k (entropy percentile)	21	15–35	0.5170–0.5232	0.0062	16	no
NF-ToN-IoT	injection scale	6	0.5–20	0.5196–0.5232	0.0037	20	yes

B.3 Consultant complementarity on the flagged set

The feedback model consults the edges the GNN model is least certain about, so what matters is not a consultant’s accuracy over the whole graph but its accuracy where it is actually asked. Table 17 gives that, at seed 42.

Table 17: Accuracy on the entropy-flagged edge set at seed 42, for the GNN model, each consultant, and the resulting feedback model.

Dataset	Consultant	GNN model	consultant	resulting feedback model
NF-UNSW-NB15	prototype	0.547	0.647	0.658
	trained head	0.547	0.795	0.779
NF-ToN-IoT	prototype	0.343	0.284	0.361
	trained head	0.343	0.648	0.628

The NF-ToN-IoT prototype row is the clearest statement of the problem. On the edges the feedback model selects, that consultant is less accurate than the model it is supposed to correct.

Table 18 gives what happens where the two disagree, the only place a consultant can change an answer.

Table 18: Among entropy-flagged edges where the consultant and the GNN model disagree, how often the consultant is right. Seed 42, at the canonical entropy percentile for each dataset.

Dataset	Consultant	flagged set		confidence-gated half	
		right	wrong	right	wrong
NF-UNSW-NB15	prototype	38	45	13	5
	trained head	55	22	17	4
NF-ToN-IoT	prototype	45	178	28	63
	trained head	129	75	79	13

The gate is doing its job for the trained head and not for the prototype. Restricting to the consultant’s own most confident half lifts the trained head from 55 against 22 to

17 against 4 on NF-UNSW-NB15 and from 129 against 75 to 79 against 13 on NF-ToN-IoT. It does not rescue the prototype on NF-ToN-IoT, where confident disagreements are wrong more than twice as often as they are right. A gate can only concentrate a consultant’s competence; it cannot manufacture it.

Both tables are recomputed from the committed pooled out-of-fold tensors, by a script that writes its own artifact:

```
scripts/close_complementarity_split.py
results/consultant_complementarity.json
```

B.4 Model comparisons and metric interpretation

Macro-F1 weights each evaluated class equally. Accuracy and weighted F1 give more influence to frequent classes, so on NF-ToN-IoT they are strongly influenced by benign traffic, which accounts for 82% of scored edges. The component comparison in Table 4 reports all three metrics, including the head alone. A high accuracy can therefore coexist with weak rare-class performance.

The feedback model’s position depends on its consultant. With the prototype scorer it is above the GNN model and the semantic model on both datasets, below the fusion model on NF-UNSW-NB15 by 0.0149, and above the fusion model on NF-ToN-IoT by 0.0419. With the trained head it is the highest of the four stages on both, but the head alone has a higher mean and its difference from feedback is not separated. The change is +0.0697 on NF-UNSW-NB15 and +0.0523 on NF-ToN-IoT. Two things changed together and the report does not separate them: the consultant, and the selected entropy percentile, which moved from 31 to 29 and from 25 to 16.

Both configurations stored their pooled out-of-fold predictions at all three seeds, so the change carries an interval computed the same way as every other difference here. It is +0.0701 [+0.0353, +0.1057] on NF-UNSW-NB15, separated, and +0.0511 [−0.0082, +0.1110] on NF-ToN-IoT, not separated. The seed-matched interval on NF-ToN-IoT does exclude zero, at +0.0509 [+0.0022, +0.1013]. That is the same pattern the fusion model against the GNN model shows below: a difference that survives resampling edges and stops surviving once training-seed variance is admitted. The interval measures the change as a whole. It does not attribute it to the consultant, because the entropy percentile moved with it.

The trained consultant was also scored standing alone. The trained head reaches 0.8374 ± 0.0048 on NF-UNSW-NB15 and 0.5081 ± 0.0073 on NF-ToN-IoT. It is separated above the GNN model on both, at +0.0946 [+0.0403, +0.1470] and +0.0725 [+0.0127, +0.1313]. The feedback model is *not* separated from it: −0.0032 [−0.0228, +0.0128] on

NF-UNSW-NB15 and -0.0032 $[-0.0342, +0.0299]$ on NF-ToN-IoT, the sign is unstable across the three seeds, and the head carries the higher mean on both. We report this because the consultant experiment is the central manipulation of this study and a reader cannot weigh it without knowing where the consultant sits on its own. It is shown as a component in Table 4, rather than as a parallel stage of the ladder, and Section 5.4 reads it against the isolation experiment.

Figure 6 and Table 5 report the stage comparisons for both consultants. Under the prototype scorer, no comparison between the feedback model and a structural rung is separated on either dataset. Every one of those intervals crosses zero, including the two where the feedback model carries the higher mean. The one prototype-consultant difference that is separated is the feedback model against the semantic model on NF-ToN-IoT, at $+0.1727$ $[+0.1326, +0.2174]$, where the semantic model scores 0.2785 against the GNN model’s 0.4336 and clearing it says nothing about the mechanism. On NF-UNSW-NB15 even that comparison crosses zero, at $+0.0287$ $[-0.0061, +0.0631]$. In that configuration the answer to RQ3 is that the ladder is an ordering of means and no more.

One result is negative regardless of consultant. The fusion model is not separated from the GNN model: positive on NF-UNSW-NB15 and negative on NF-ToN-IoT, with an interval crossing zero either way. The seed-matched interval for NF-UNSW-NB15 does exclude zero, $+0.0356$ $[+0.0038, +0.0678]$. That is the clearest illustration in this report of what the two-level procedure is for: once seed variance enters, the same difference stops being separated. Under the pre-registered rule, the fusion model versus the GNN model is not separated, and the answer to RQ2 is negative on both datasets.

B.5 Baseline scoring asymmetry and comparison details

The feedback comparisons below remain subject to the component check: the trained head alone has a higher mean macro-F1 on both datasets, with no separated difference from feedback (Table 4).

The collapse of E-GraphSAGE on NF-UNSW-NB15 is a representation effect with a testable prediction attached, not evidence that the architecture is weak. Its edge classifier reads only the concatenation $[h_u || h_v]$ of the two endpoint embeddings. An edge’s own attributes never reach it, so parallel edges between one address pair cannot be told apart. That predicts a collapse proportional to how common parallel edges are. On NF-UNSW-NB15, 385 of 656 edges are indistinguishable by endpoints, that is 58.7%, and the architecture scores 0.1194. On NF-ToN-IoT only 167 of 2,127 edges are, that is 7.9%, and the same architecture scores 0.4141. The prediction and the inversion agree. Figure 8 shows the mechanism beside the confirmation. The ceiling is also not a

training-schedule artifact: trained to the authors’ full 4,999 epochs on fold 0, validation macro-F1 plateaus near 0.11 and peaks at 0.1384 at epoch 1,400.

The lower rungs are weaker still. On NF-ToN-IoT neither the GNN model nor the fusion model is separated from E-GraphSAGE as published, at $+0.0203$ $[-0.0230, +0.0633]$ and -0.0061 $[-0.0797, +0.0777]$. On NF-UNSW-NB15 neither the GNN model nor the semantic model is separated from TE-G-SAGE with our node features, at $+0.0249$ and $+0.0163$.

The NF-ToN-IoT baselines were scored under a looser rule than the rungs they are compared against, and the margin has to be read against that. As Section 3.9 states, these models and the fusion model could predict **dos** or **ransomware** for an edge the metric scores, losing it, while the GNN, semantic and feedback models could not. Across the three seeds that happened on 3, 374 and 6 edges for the fusion model, on 58, 63 and 47 for E-GraphSAGE as published, on 66, 28 and 60 for E-GraphSAGE with our node features, and on between 29 and 206 for the three TE-G-SAGE configurations. Replacing every one of those predictions with the true label gives an upper bound on what the mask could have been worth, and that bound is more generous than any mask could be, because no mask redirects an edge to the right class. The bound stays below the feedback model at every seed for the fusion model and for five of the six baselines. The exception is E-GraphSAGE with our node features at seed 1, where it reaches 0.5206 against the feedback model’s 0.5012. Table 9 already reports that comparison as not separated, at $+0.0667$ $[-0.0181, +0.1505]$, so no claim in this report depends on the asymmetry. Every count and both macro-F1 values, per model and per seed, are in `results/ton_iot_dropped_class_bound.json`.

Because most of the NF-UNSW-NB15 margin is not architectural, it repays decomposition. Table 10 walks from the strongest published configuration to our rungs. Every level is a three-seed mean, no step carries an interval, and no step is a separation claim.

Figure 9 draws the walk. Roughly 45% of the naive gap is our choice of how to tune, roughly 30% is our node features, and roughly 25% is the architecture. Measured to the fusion model instead of the feedback model, the architecture share is 15.1%.

C Reproducibility

This appendix first records the software environment and execution settings, then provides the reproduction commands, and finally identifies the supporting artifacts.

Environment. Python 3.14.3 in a virtual environment built from `requirements.txt`. Message passing, the fold masks and the graph containers come from PyTorch Geometric [9]. The stratified fold generator and every reported metric are scikit-learn’s [24]. The ZCA whitener is a local eigendecomposition rather than a library call. All runs export `OMP_NUM_THREADS=1`. Without that pin, pooled macro-F1 drifts by roughly 0.012 at a fixed seed, larger than several of the differences this report measures. One earlier finding in the project was retracted because of it.

Computational cost. Everything in this report was produced on one Apple M1 Pro laptop with 8 cores, using the MPS backend for semantic encoding and CPU for everything else. Semantic encoding is one measured per-edge inference cost, requiring a frozen forward pass for each edge: at batch size 32 and a 128-token limit it encodes 171.2 edges per second on NF-UNSW-NB15 and 160.2 on NF-ToN-IoT, so 5.841 and 6.241 milliseconds per edge, and all 656 and 2,127 edges in 3.83 and 13.28 seconds. Loading the encoder costs a further 3.2 seconds once per process. Training is an offline cost paid once: one five-fold run of the fusion stage at one seed takes 6.4 seconds on NF-UNSW-NB15 and 17.3 on NF-ToN-IoT. These are graphs of a few hundred to a few thousand edges, so the figures say nothing about how the encoder would scale to per-flow traffic volumes; they establish that nothing in this study was compute-bound. Measurements are in `results/encoding_cost.json`.

Seeds and folds. Three training seeds, 42, 1 and 2. The stratified five-fold partition is generated once at seed 42 and held fixed across all three, so the pooled out-of-fold edge set is identical between seeds and the seed-matched bootstrap is a paired comparison. Fold-partition variance is therefore not measured anywhere in this report.

Pipeline. Graph construction runs from `src.pipeline.step1.run_step1`, writing `pyg_data.pt` and `aggregated_edges.csv` under `data/<dataset>/processed/step1/`. The downstream stages run through one entry point:

```
python -m src.pipeline.common.run_pipeline -dataset <key> \
-only fusion oof prototypes top_k_sweep feedback ladder
```

with `<key>` either `unsw_nb15` or `ton_iot`. That runner passes `-use-llm-head` to the sweep and feedback stages, so the feedback model consults the trained head as the contracts describe. Calling `train_feedback` without that flag reproduces the superseded prototype-consultant ladder instead, at the knobs kept under `prototype_superseded` in each dataset’s selected-feedback config. The runner’s `-seed` is the training seed and reaches every stage that trains; the fold partition has its own `-split-seed`, which defaults to 42 and should not be changed if the results are to stay comparable with the ones reported here. Out-of-fold structural embeddings must be regenerated with `step3.build_oof_gnn_embeddings` before the fusion stage whenever the GNN model changes. The fusion stage does not do it, and will otherwise train on stale embeddings.

`reproduce_ladder.py` rebuilds the NF-UNSW-NB15 ladder end to end and compares it against the committed contract, printing any rung that differs by more than 0.0005.

Result contracts. Every number in this report traces to one of the files in Table 19.

Table 19: Committed artifacts behind the report’s numbers.

Artifact	What it carries
<code>results/unsw_nb15_current.json</code> (schema 5)	UNSW ladder, ablations, knobs
<code>results/ton_iot_current.json</code> (schema 7)	ToN ladder, ablations, knobs
<code>results/cross_dataset_comparison.json</code> (schema 6)	both datasets side by side
<code>results/multiseed_ladder_v2_head.json</code>	three-seed ladder, trained-head consultant
<code>results/multiseed_ladder_v2_legacy.json</code>	three-seed ladder, prototype
<code>results/multiseed_head_per_class.json</code>	per-class F1, three-seed means
<code>results/*_sota_baselines.json</code>	re-trained baselines and intervals
<code>results/*_oracle_ceiling_v2_trained_head.json</code>	the isolation test
<code>results/knob_selection_head/</code>	the selection curves
<code>results/consultant_change_interval.json</code>	the interval on the consultant change
<code>results/consultant_complementarity.json</code>	the flagged-set and disagreement figures
<code>results/ton_iot_dropped_class_bound.json</code>	the NF-ToN-IoT scoring asymmetry, bounded
<code>docs/RESULTS_ARCHIVE.md</code>	the full record, including retractions

Checks and data access. The recorded test run passed 174 tests and 359 subtests with `OMP_NUM_THREADS=1`. The build regenerates the tables and runs `scripts/check_report_numbers.py`; its exit status should also be checked directly because the build treats audit failures as warnings. Writing statistics are available separately through `scripts/prose_stats.py`. Raw dataset CSVs are not included in the repository. Recalculating the churn diagnostic additionally requires the local traces identified in Appendix A.5.